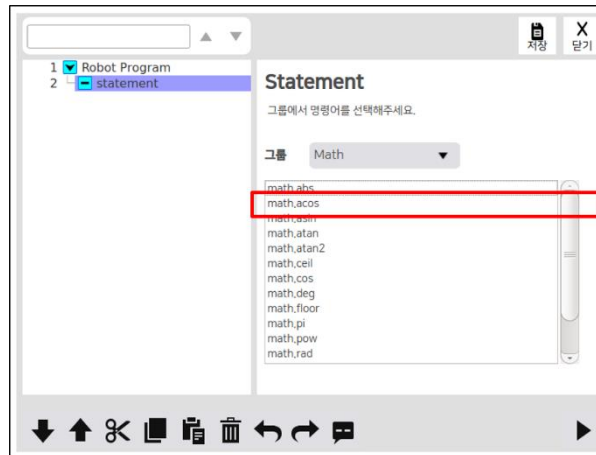
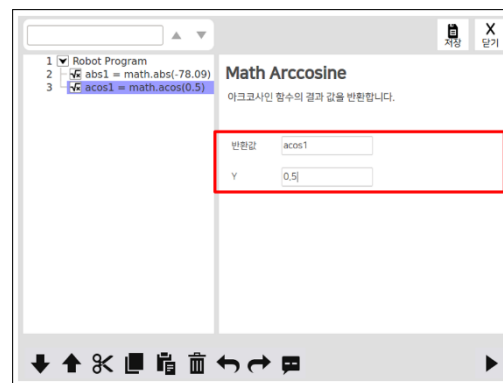
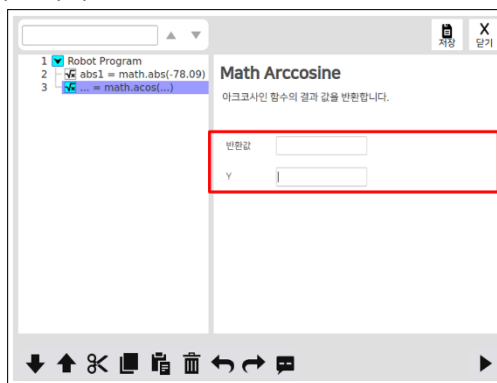


2. Arc cosine

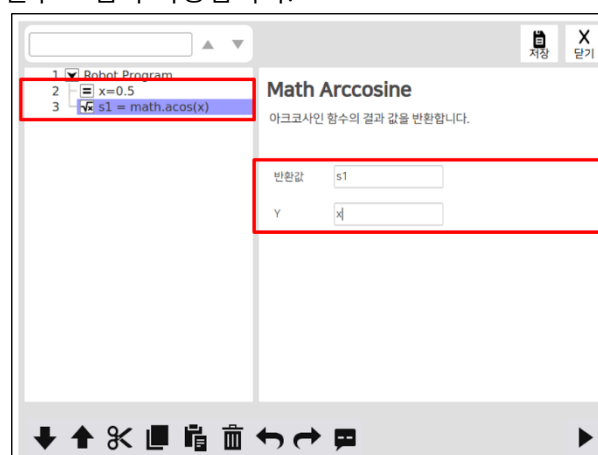
- `math.acos` 은 아크코사인 함수의 결과값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.acos` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 Y 값의 아크코사인 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

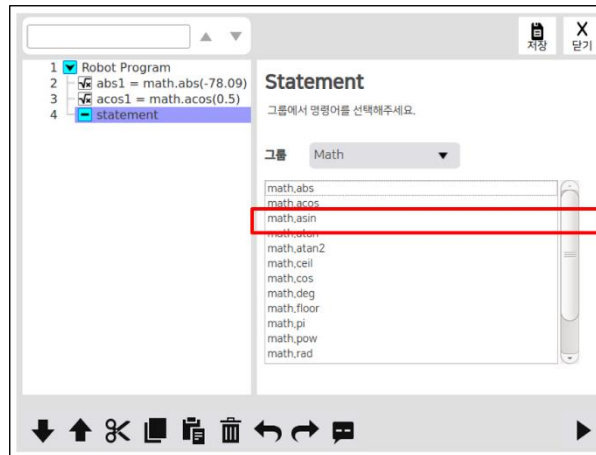


- Y 칸에 아크코사인 값으로 반환될 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 Y 값은 선언되어 있는 변수로 입력 가능합니다.

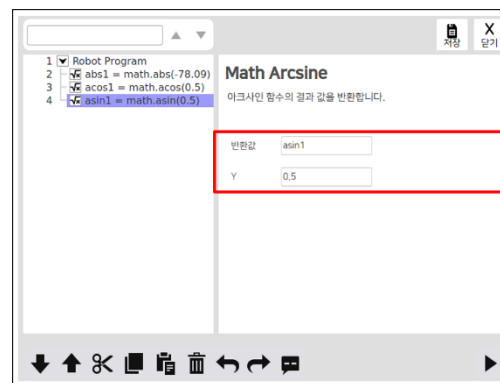
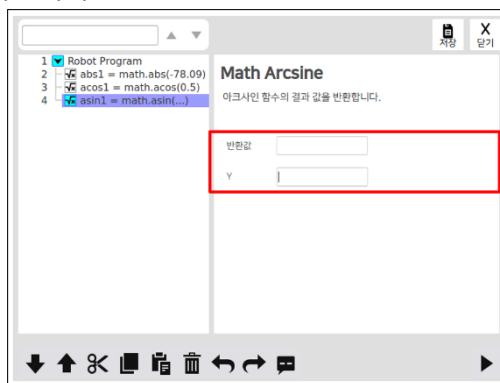


3. Arc sine

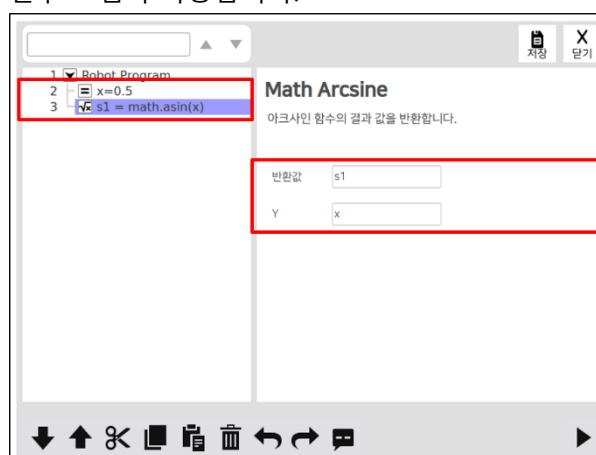
- `math.asin` 은 아크사인 함수의 결과값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.asin` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 Y 값의 아크사인 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

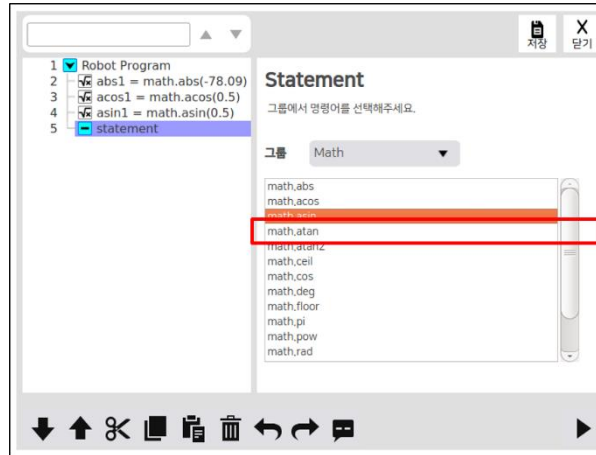


- Y 칸에 아크사인 값으로 반환될 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 Y 값은 선언되어 있는 변수로 입력 가능합니다.

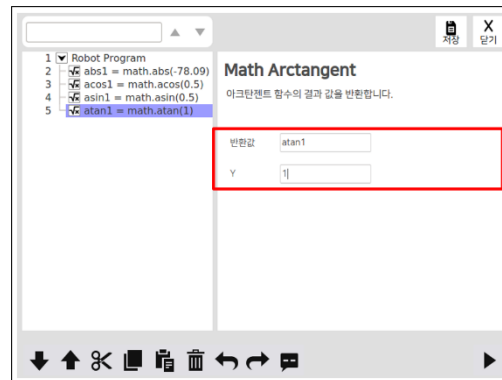
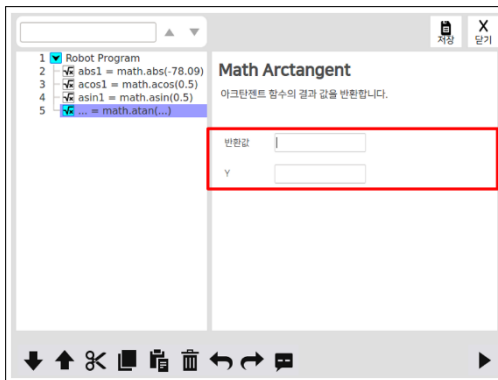


4. Arc tangent

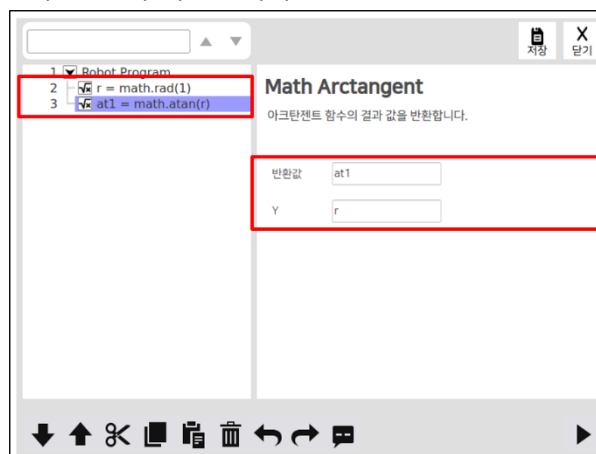
- `math.atan` 는 아크탄젠트 함수의 결과값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.atan` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 Y 값의 아크탄젠트 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

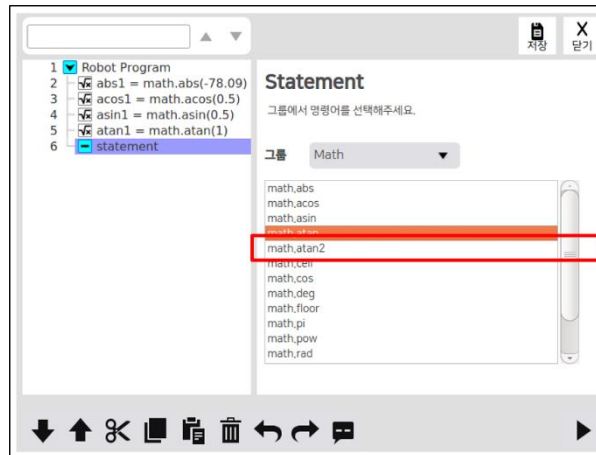


- Y 칸에 아크사인 값으로 반환될 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 Y 값은 선언되어 있는 변수로 입력 가능합니다.

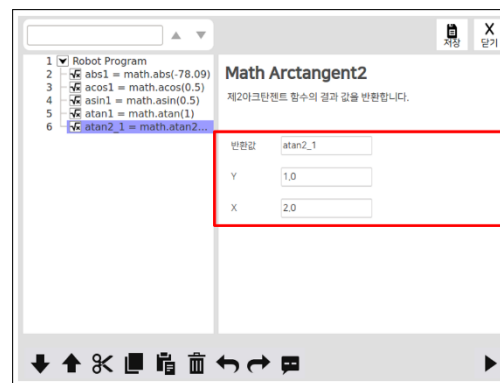
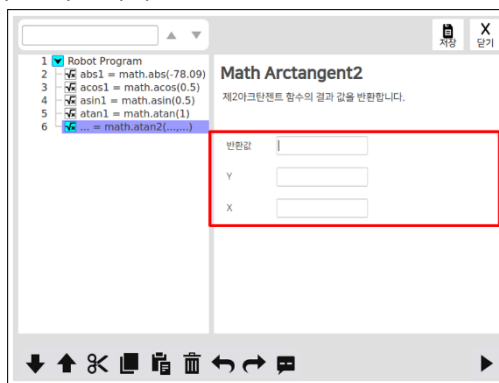


5. Arc tangent2

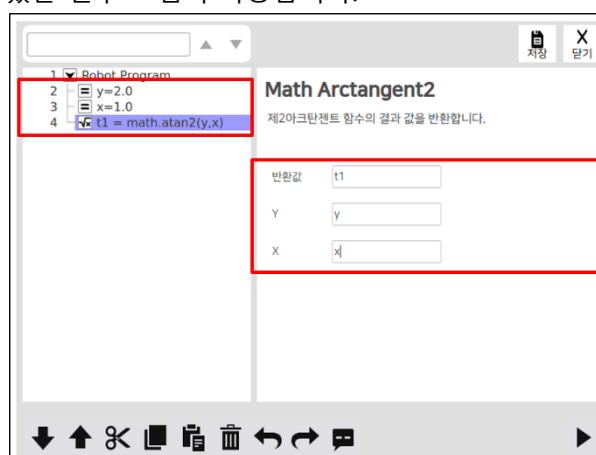
- `math.atan2` 는 제 2 아크탄젠트 함수의 결과값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.atan2` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X,Y 값의 제 2 아크탄젠트 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

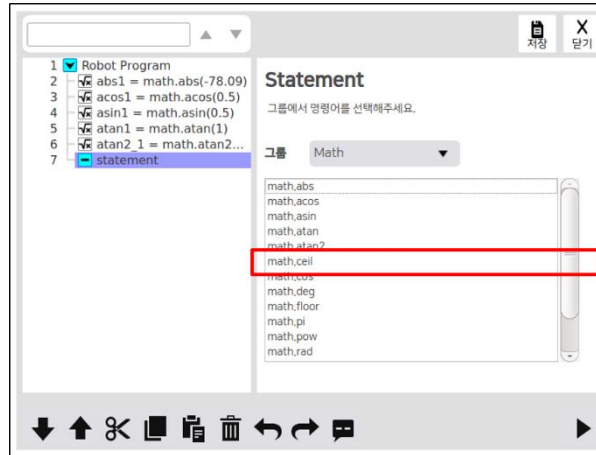


- X,Y 칸에 직각 삼각형의 너비 및 높이 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X,Y 값은 선언되어 있는 변수로 입력 가능합니다.

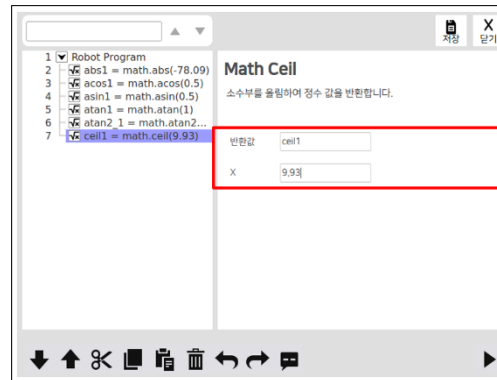
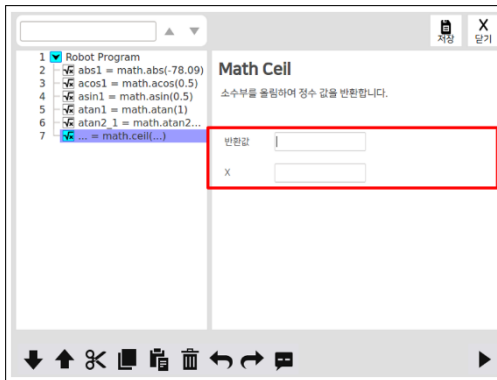


6. Ceil

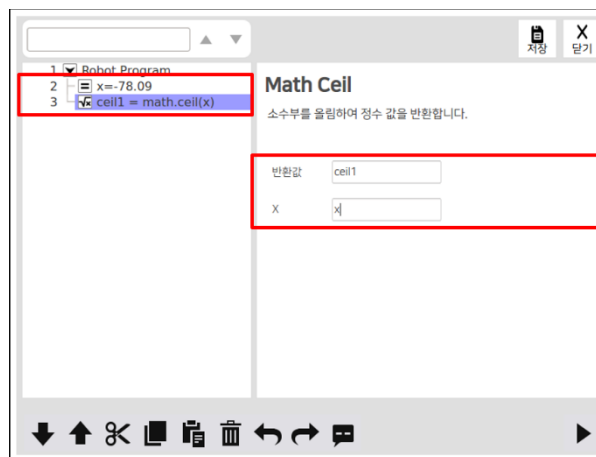
- `math.ceil` 은 소수부를 올림하여 정수 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.ceil` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 올림값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

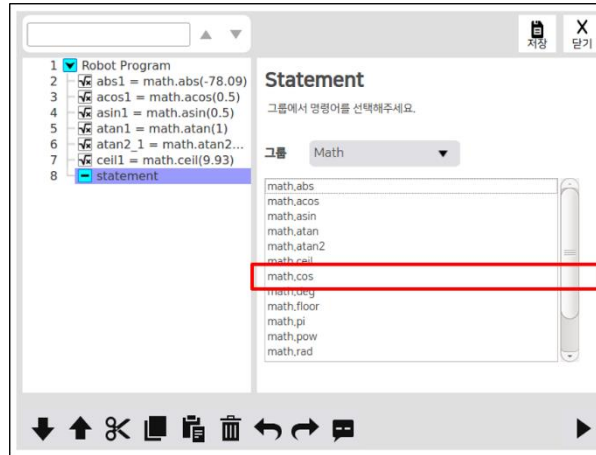


- X 칸에 실수형 입력 인자 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

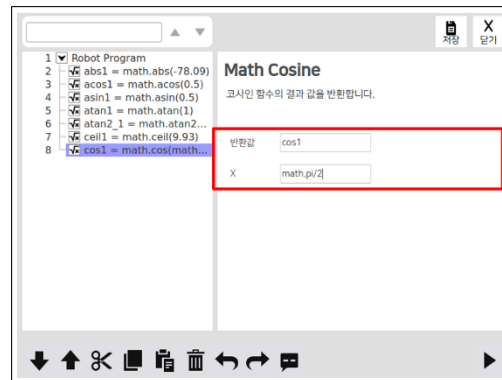
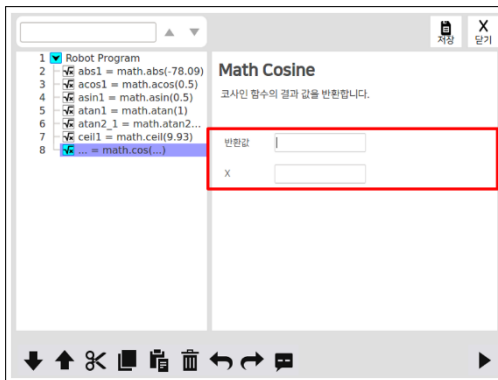


7. Cosine

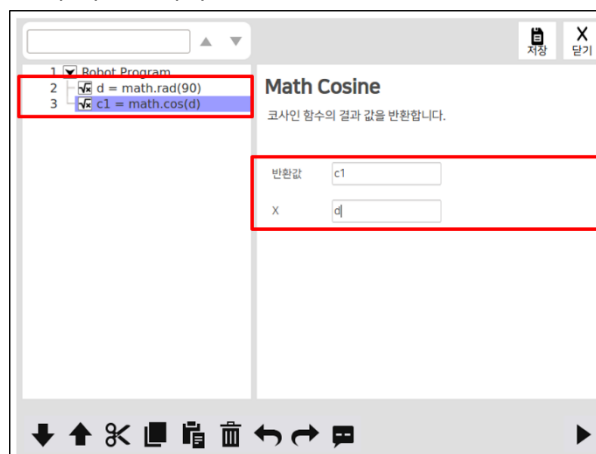
- `math.cos` 은 코사인 함수의 결과 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.cos` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 코사인 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

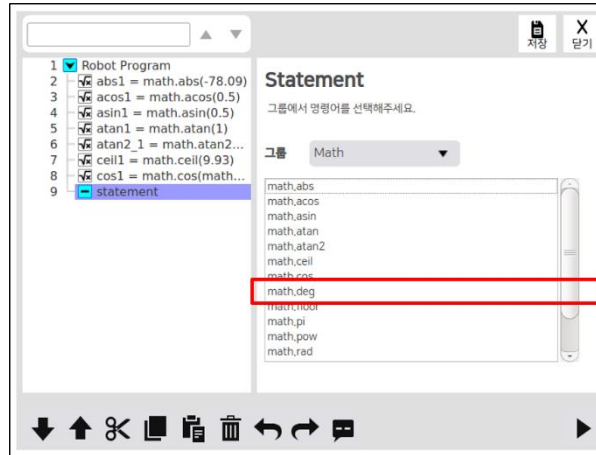


- X 칸에 입력 radian 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

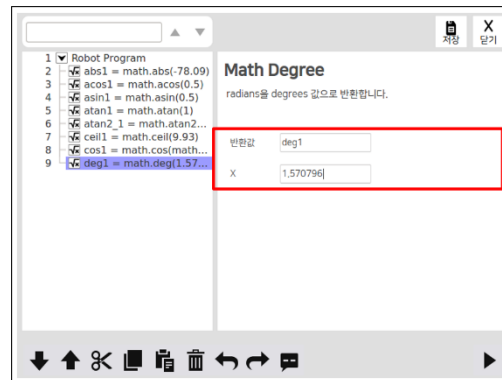
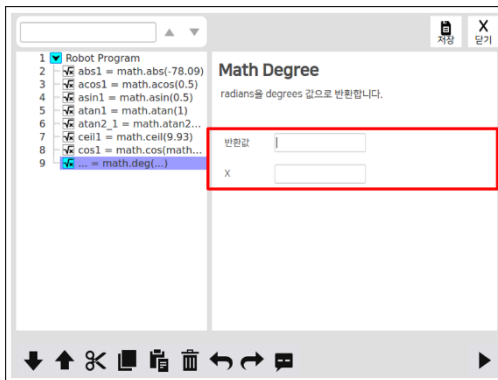


8. Degree

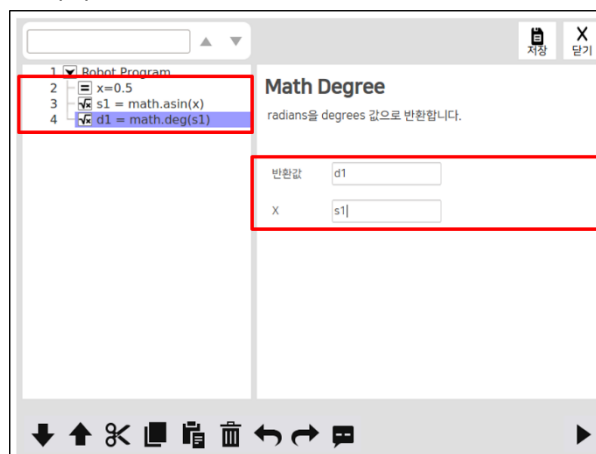
- `math.deg` 는 radian 을 degree 값으로 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.deg` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 degree 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

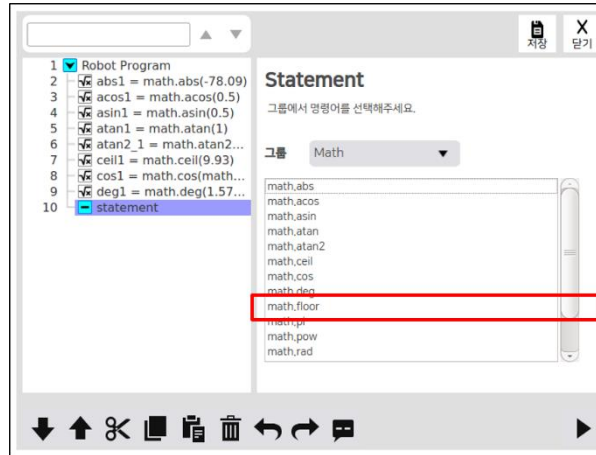


- X 칸에 radian 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

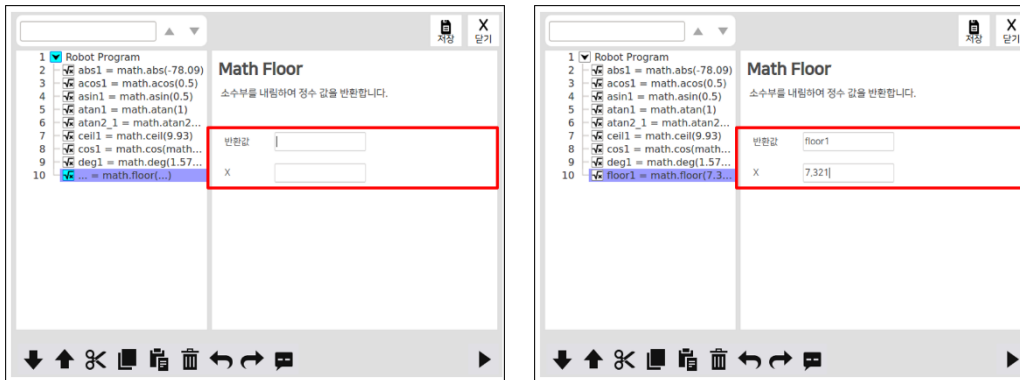


9. Floor

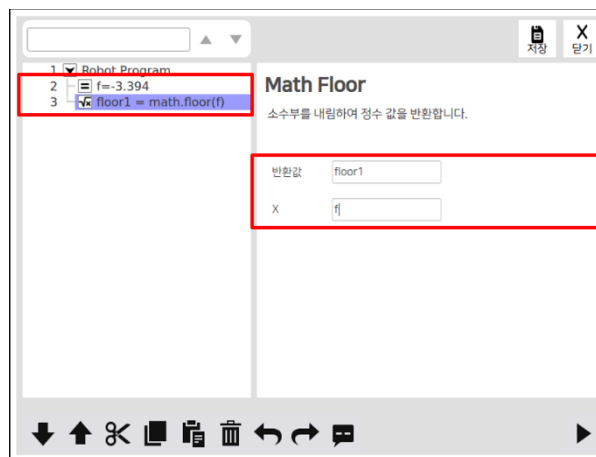
- `math.floor` 는 소수부를 내림하여 정수 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.floor` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 내림값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

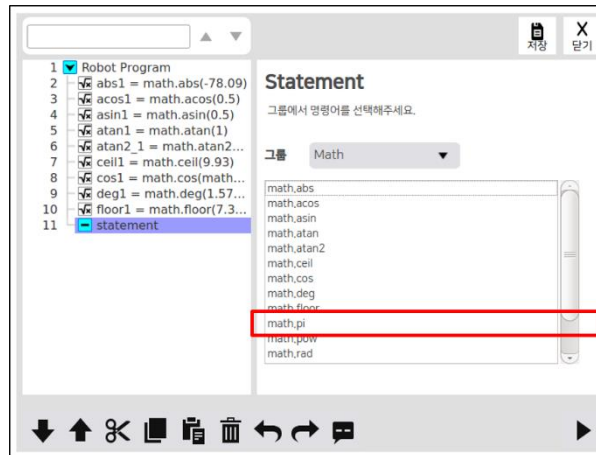


- X 칸에 실수형 입력 인자 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

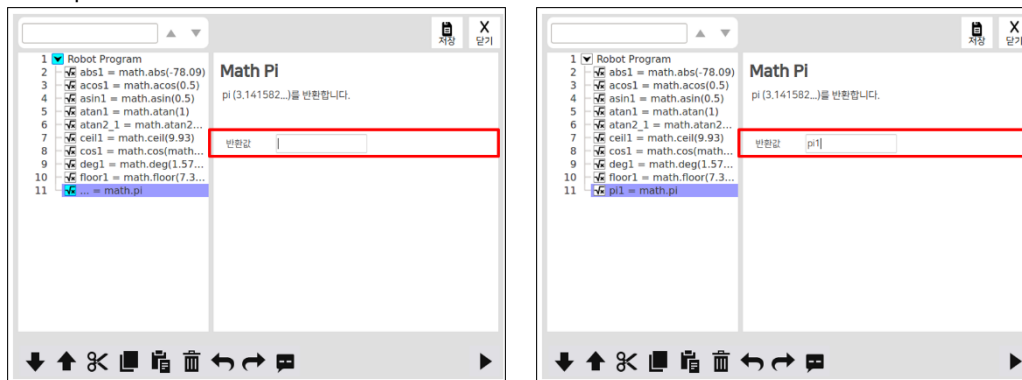


10. Pi

- `math.pi` 는 `pi(3.141582...)`를 반환하는 명령어 입니다.

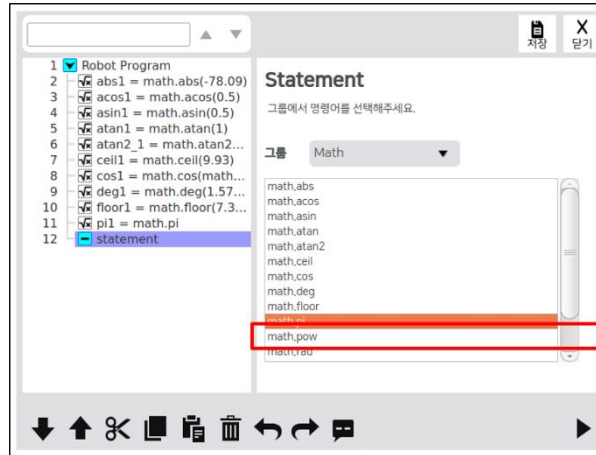


- Statement 의 Math 그룹에 있는 `math.pi` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 pi 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

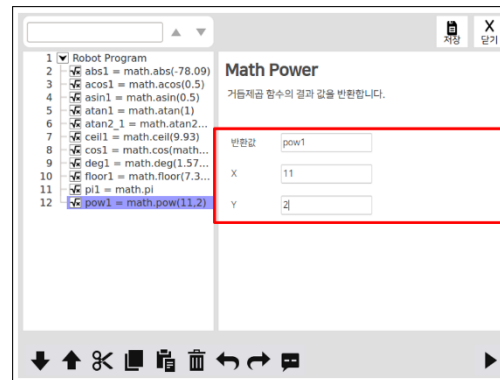
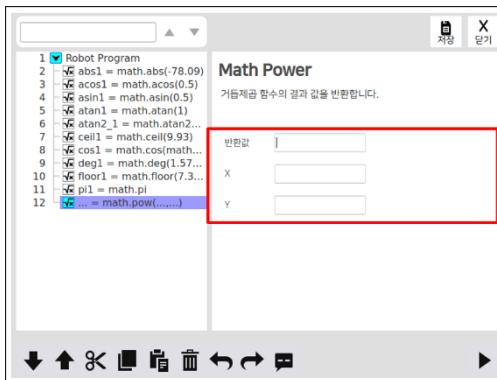


11. Power

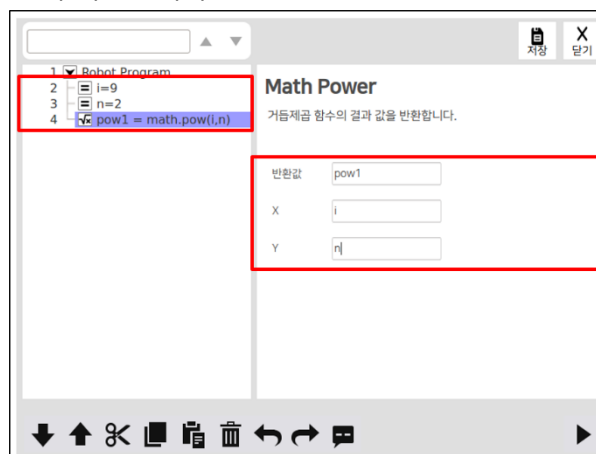
- `math.pow` 는 거듭제곱 함수의 결과 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.pow` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 거듭 제곱 입력 인자값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

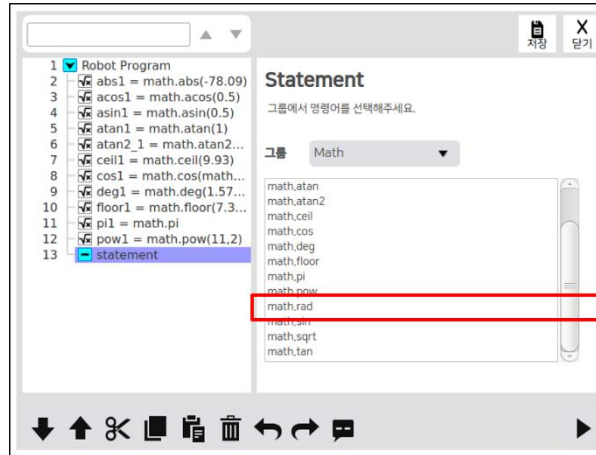


- X 칸은 밑수, Y 칸은 지수 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X,Y 값은 선언되어 있는 변수로 입력 가능합니다.

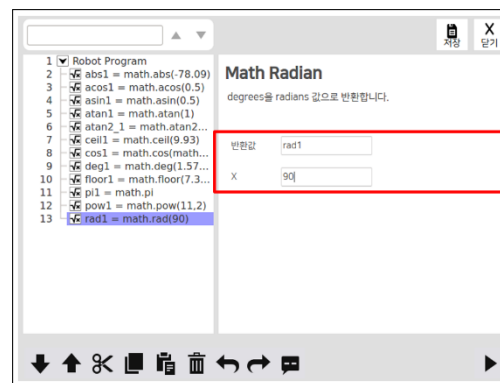
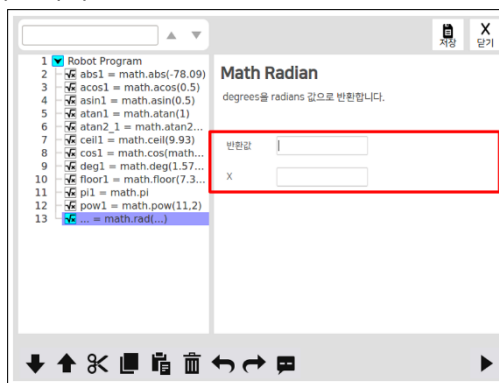


12. Radian

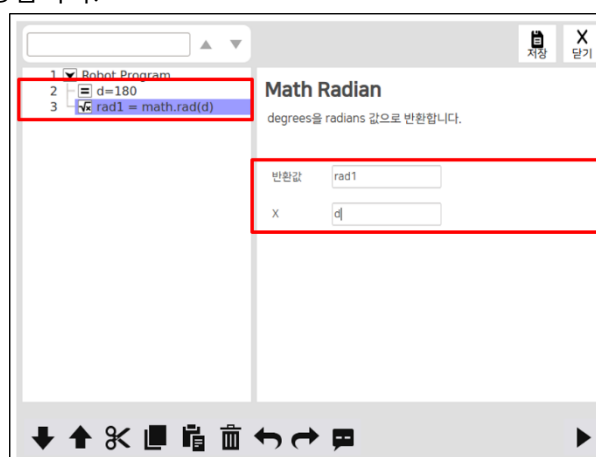
- `math.rad` 은 degree 를 radian 값으로 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.rad` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 radian 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

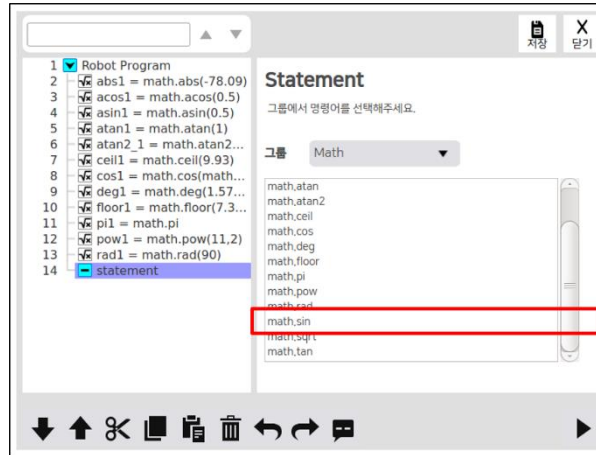


- X 칸은 degree 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

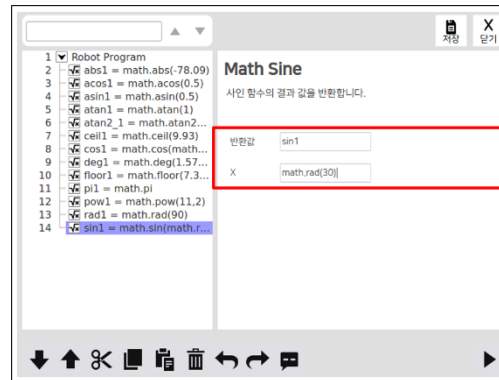
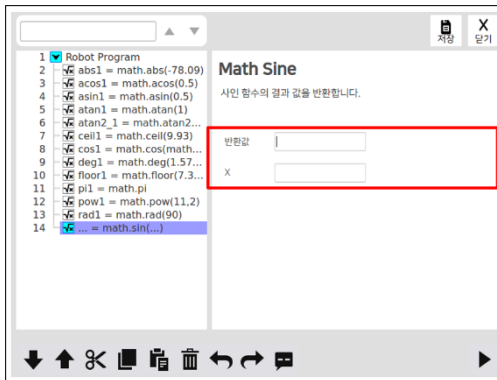


13. Sine

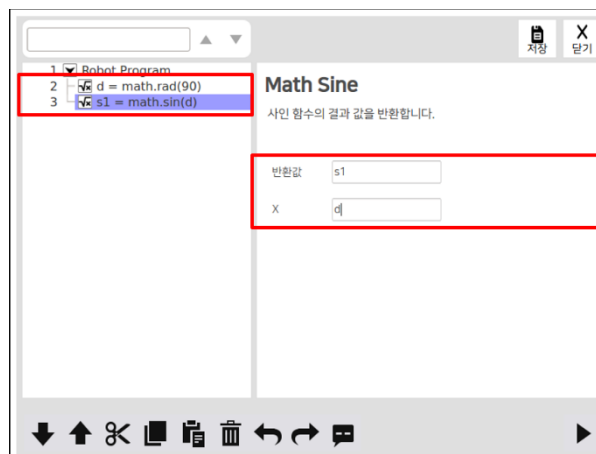
- `math.sin` 은 사인 함수의 결과 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.sin` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 사인값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

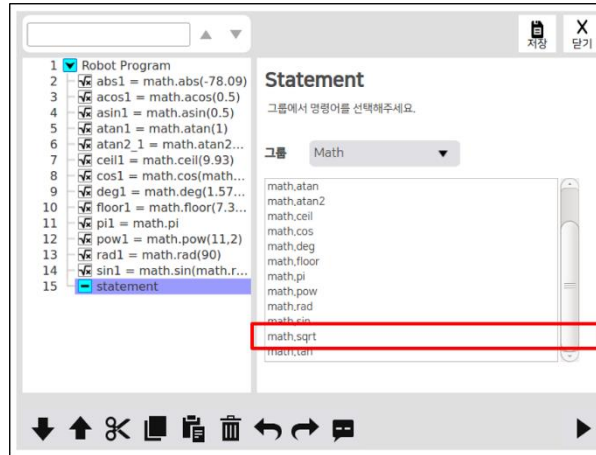


- X 칸은 radian 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

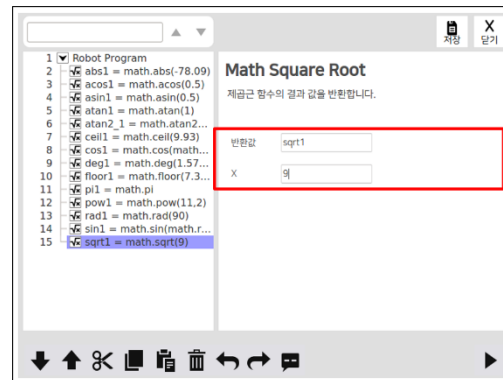
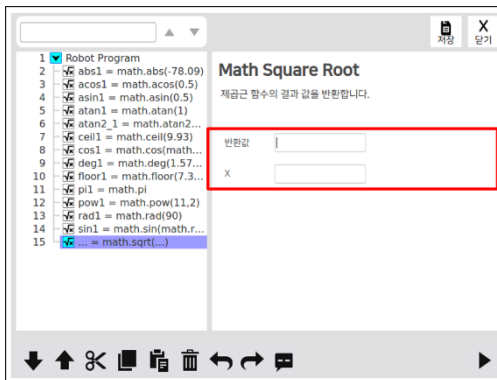


14. Sqrt

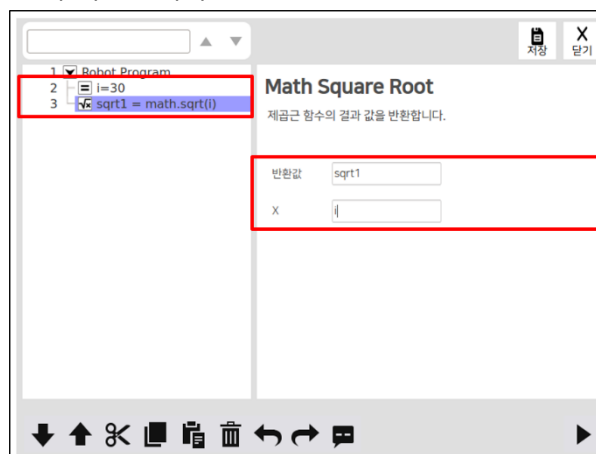
- `math.sqrt` 는 제곱근 함수의 결과 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.sqrt` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 제곱근 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

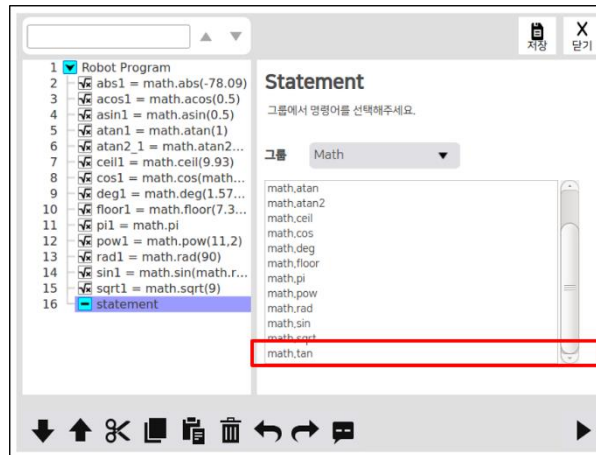


- X 칸은 제곱근 식 입력값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

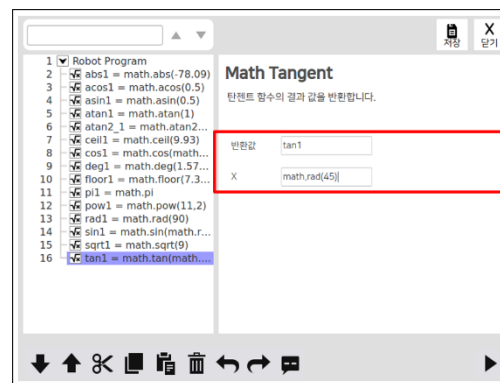
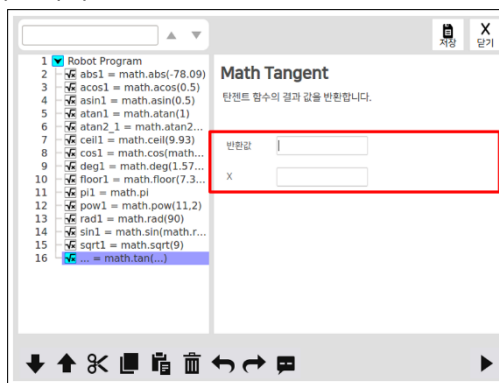


15. Tangent

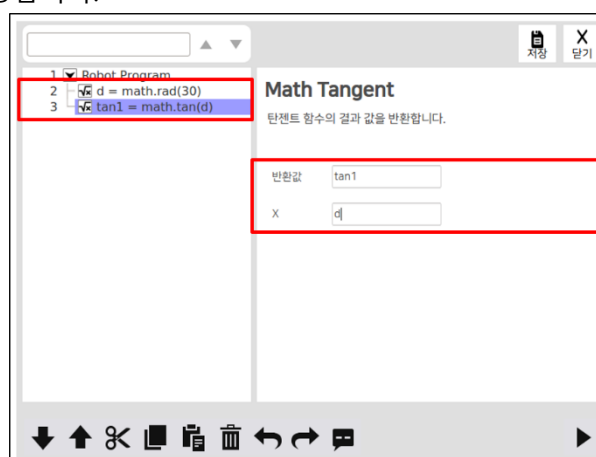
- `math.tan` 는 탄젠트 함수의 결과 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 `math.tan` 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 탄젠트 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.



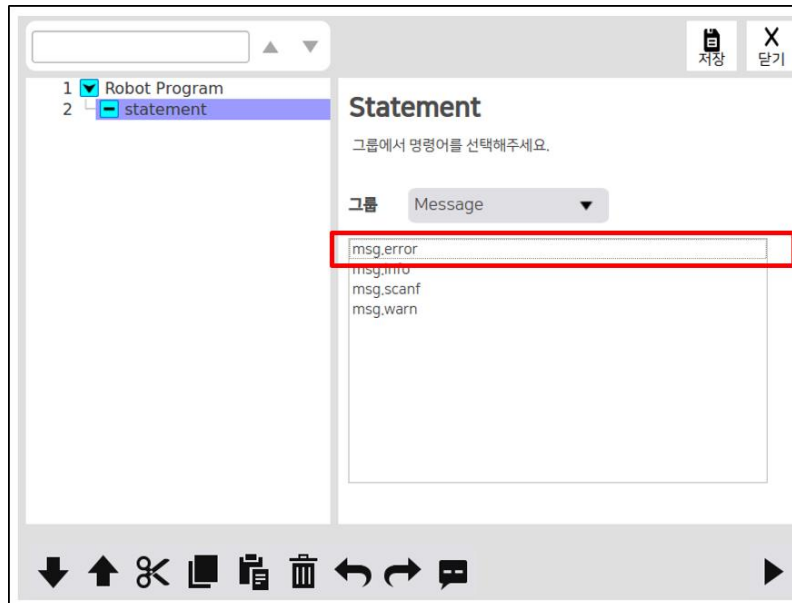
- X 칸은 radian 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.



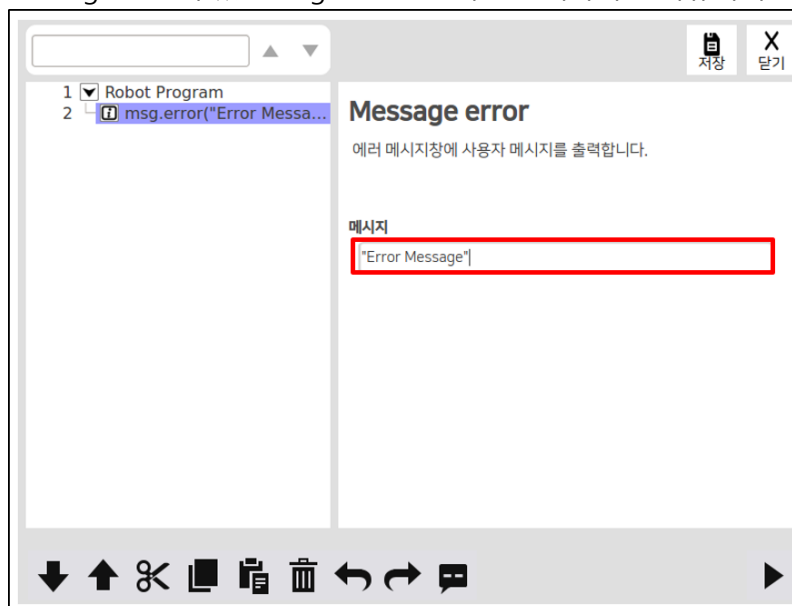
1.1.1.3.5 Message

1. Error

- msg.error 는 에러 메시지창에 사용자 메시지를 출력하는 명령어 입니다.



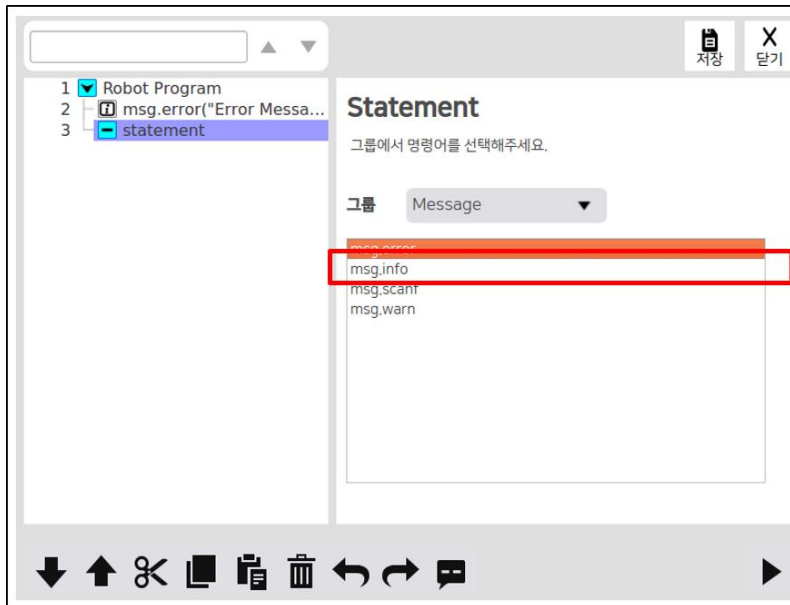
- Statement 의 Message 그룹에 있는 msg.error 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



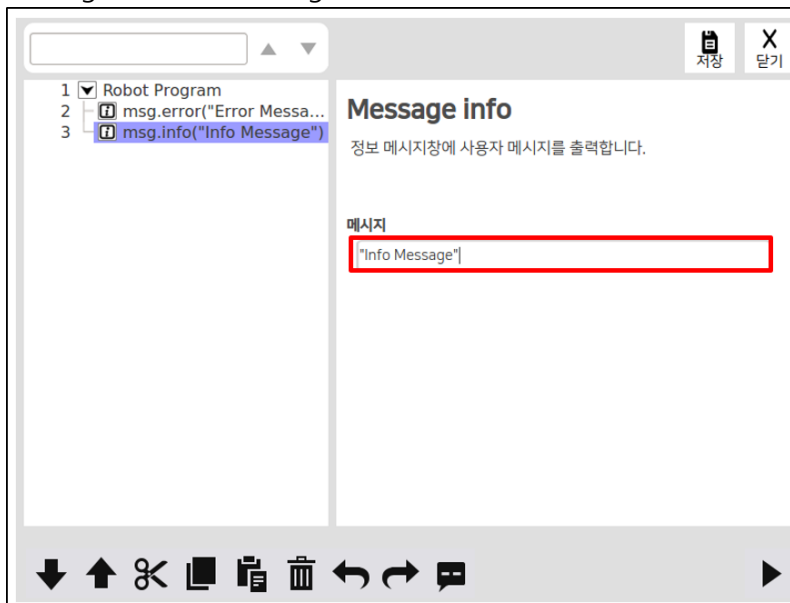
- 메시지 입력 칸을 클릭하면 키보드가 팝업됩니다. 키보드를 통해 오류 메시지를 입력합니다.
- 메시지 입력 시, ""(쌍따옴표) 사이에 메시지 내용을 입력합니다.

2. Info

- msg.info 는 정보 메시지창에 사용자 메시지를 출력하는 명령어 입니다.



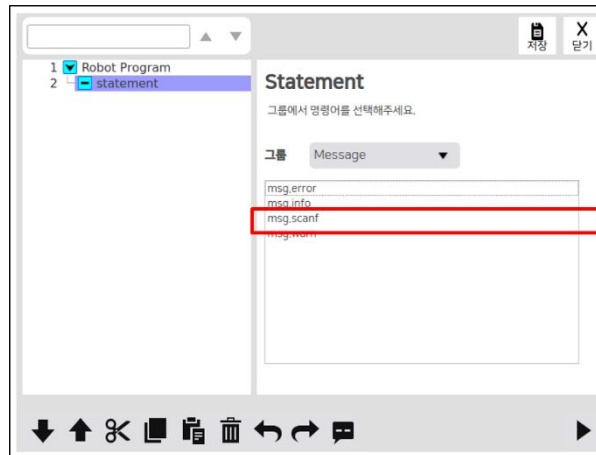
- Statement 의 Message 그룹에 있는 msg.info 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



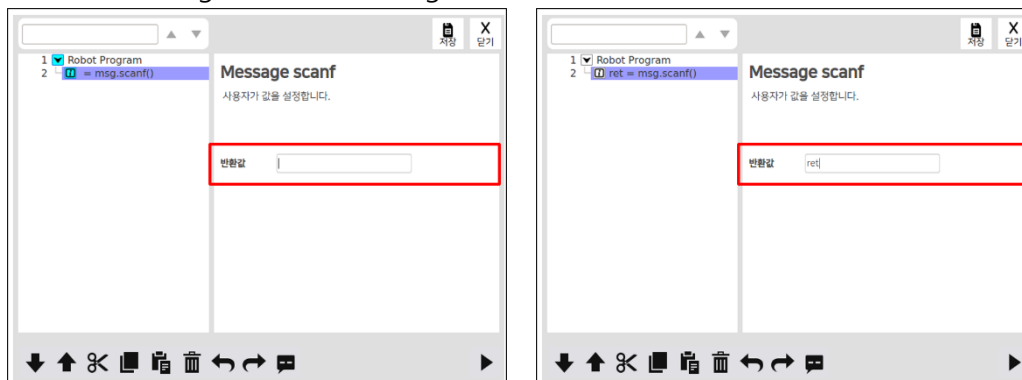
- 메시지 입력 칸을 클릭하면 키보드가 팝업됩니다. 키보드를 통해 정보 메시지를 입력합니다.
- 메시지 입력 시, ""(쌍따옴표) 사이에 메시지 내용을 입력합니다.

3. Scanf

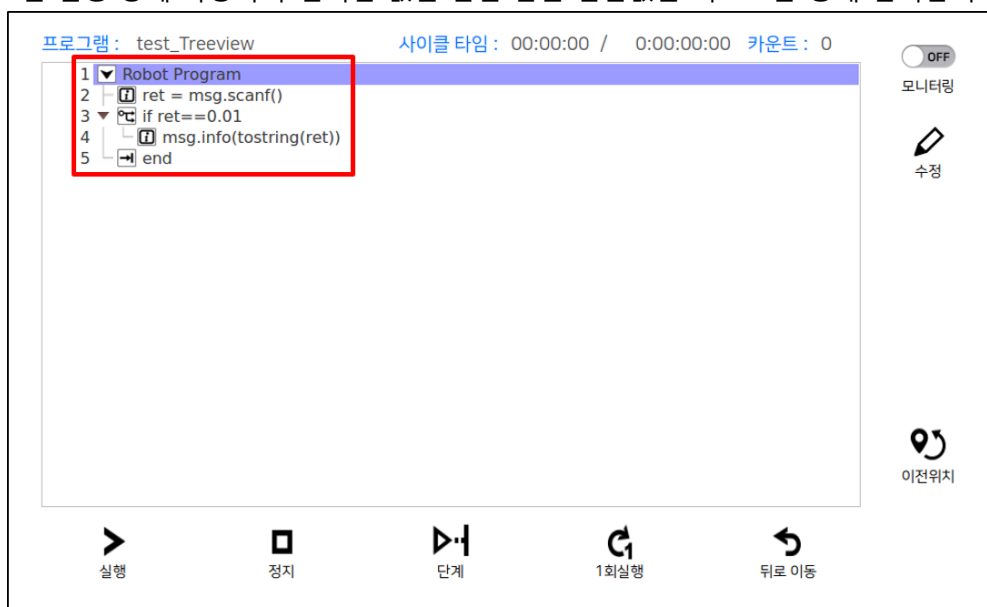
- msg.scanf 는 프로그램 실행 중에 사용자가 값을 입력하는 명령어 입니다.



- Statement 의 Message 그룹에 있는 msg.scanf 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- 프로그램 실행 중에 사용자가 입력한 값을 반환 받을 반환값을 키보드를 통해 입력합니다.



- 프로그램 제어 화면으로 이동하여 프로그램을 실행합니다.

Scanf

값

확인

Scanf

값

확인

- 프로그램을 실행하면 scanf 가 있는 라인에서 값을 입력하는 화면이 팝업됩니다. 값을 입력하고 '확인' 버튼을 클릭합니다.

Information

Info

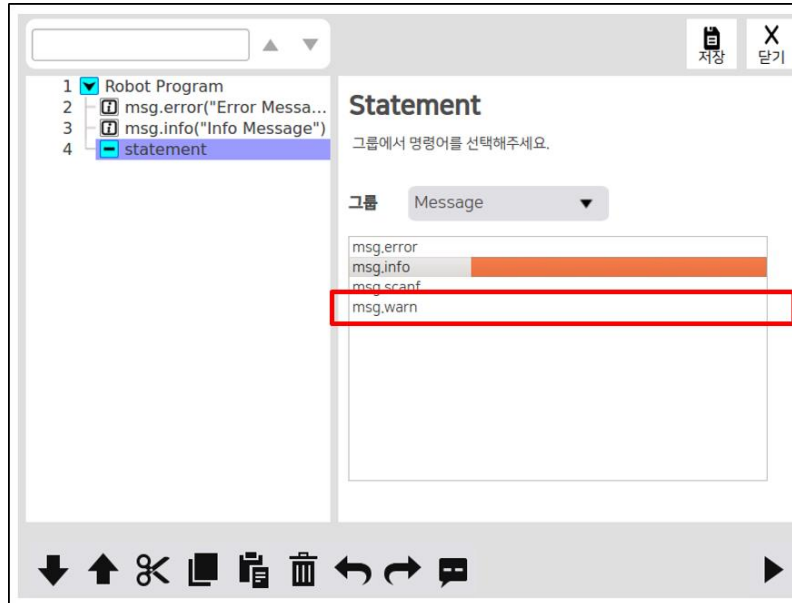
0.01

✓ 확인

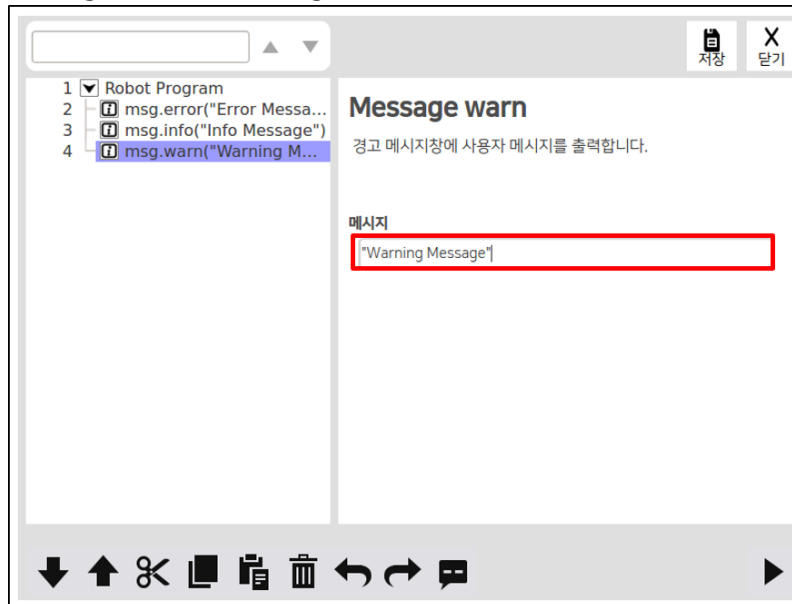
- 입력한 값에 대한 코드가 실행됩니다.

4. Warn

- msg.warn 은 경고 메시지창에 사용자 메시지를 출력하는 명령어 입니다.



- Statement 의 Message 그룹에 있는 msg.warn 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.

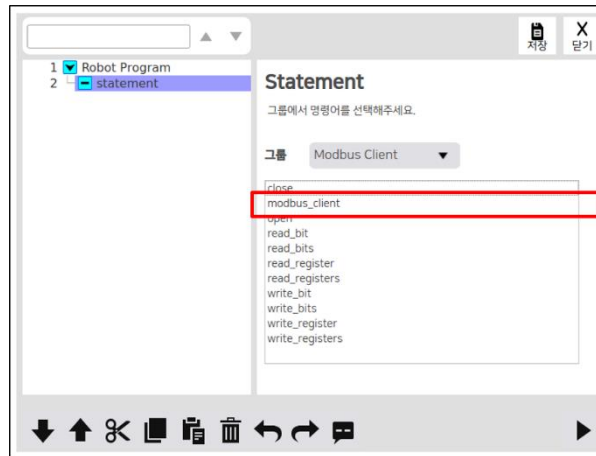


- 메시지 입력 칸을 클릭하면 키보드가 팝업됩니다. 키보드를 통해 경고 메시지를 입력합니다.
- 메시지 입력 시, ""(쌍따옴표) 사이에 메시지 내용을 입력합니다.

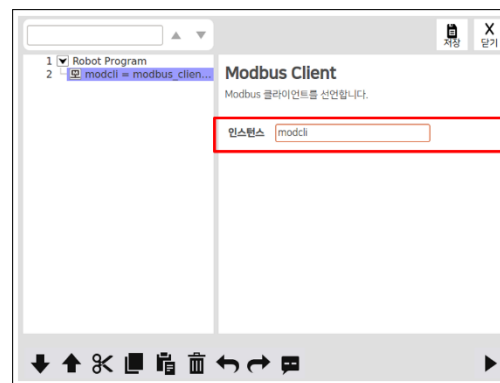
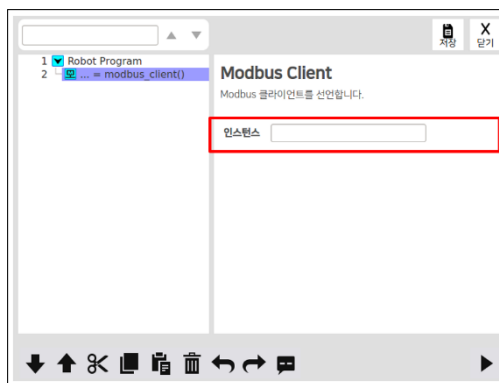
1.1.1.3.6 Modbus Client

1. Modbus_client()

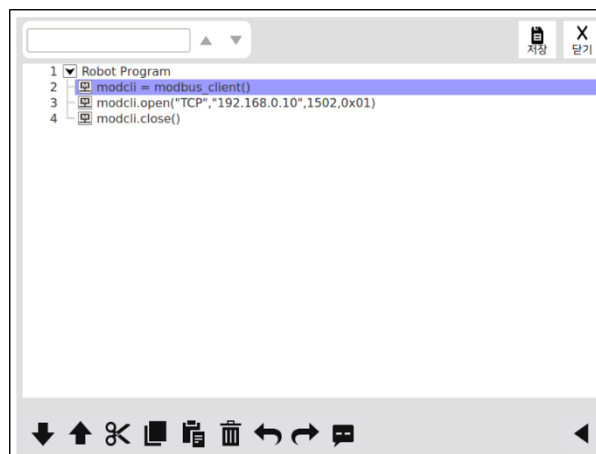
- modbus_client 는 모드버스 클라이언트를 선언하는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 modbus_client 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



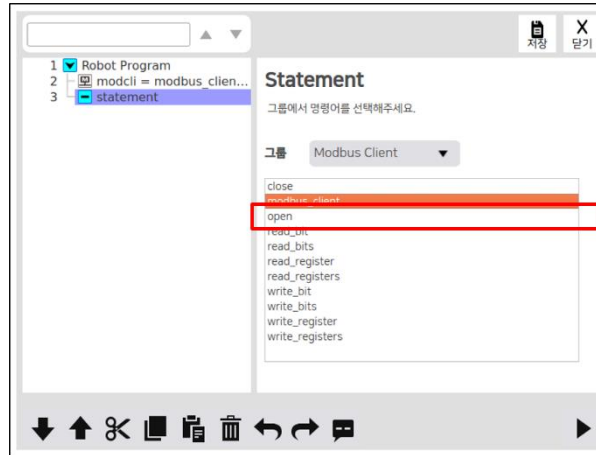
- 모드버스 클라이언트를 선언할 인스턴스 명을 입력해야 합니다. 인스턴스 칸을 클릭하여 키보드를 통해 인스턴스 명을 입력합니다.



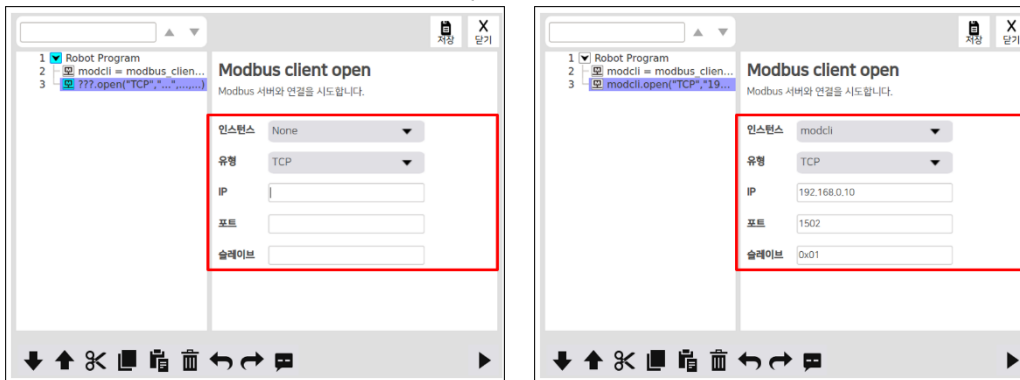
- client() 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

2. Open

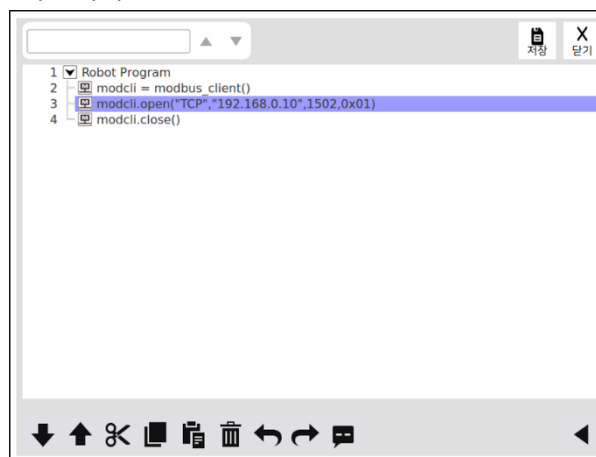
- open 은 모드버스 서버와 연결을 시도하는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 open 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



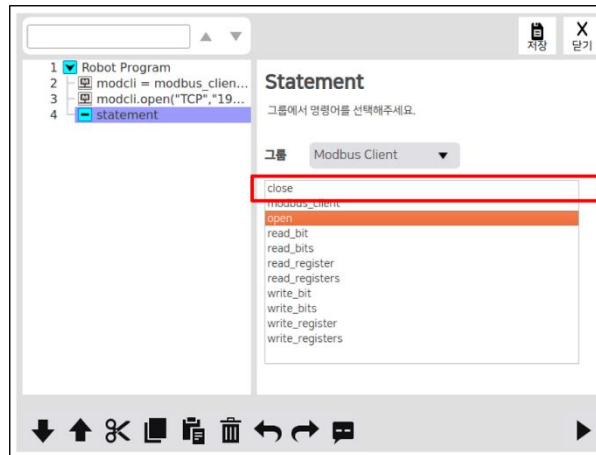
- 모드버스 서버와 연결을 시도할 인스턴스와 통신 타입, 서버 주소, 통신 포트, 슬레이브 ID 를 입력해야 합니다. 인스턴스 및 유형은 콤보 박스를 통해 선택하고 서버 주소, 통신 포트와 슬레이브 ID 는 키보드를 통해 입력합니다.



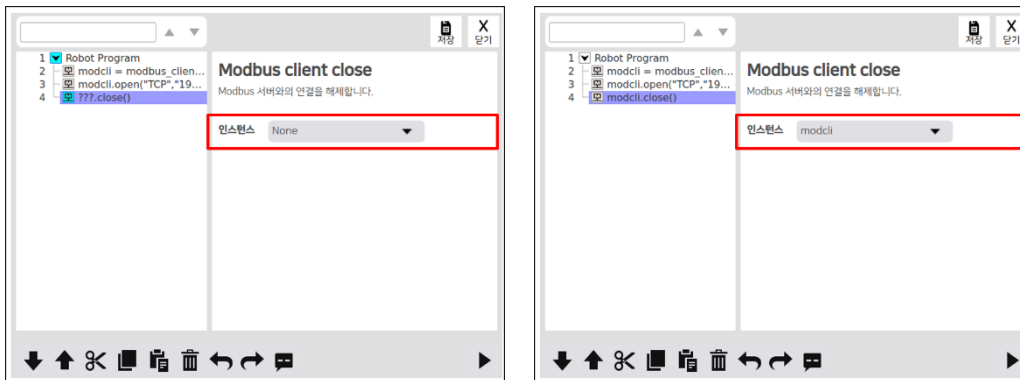
- open 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

3. Close

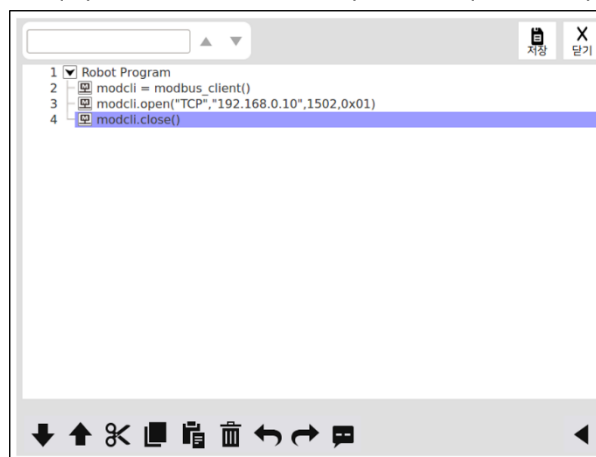
- close 는 모드버스 서버와의 연결을 해제하는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 close 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



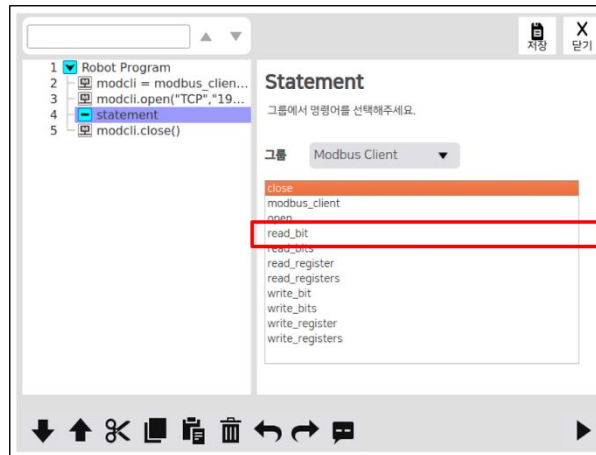
- 모드버스 서버와 연결을 해제할 인스턴스를 콤보 박스를 통해 설정합니다.



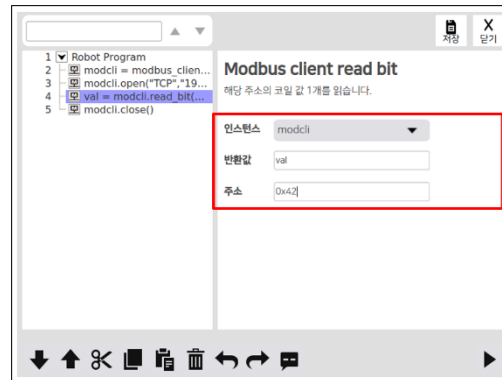
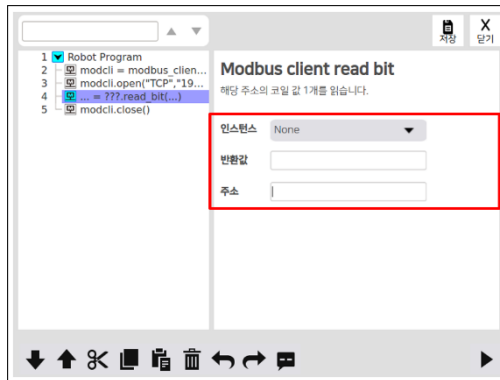
- close 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

4. Read_bit

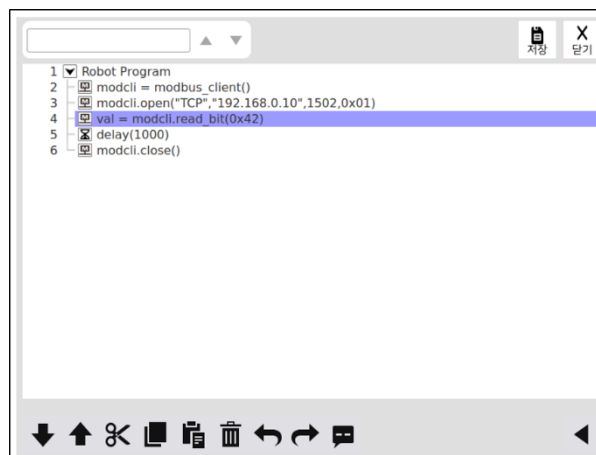
- read_bit 는 설정 주소의 코일 값 1 개를 읽는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 read_bit 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



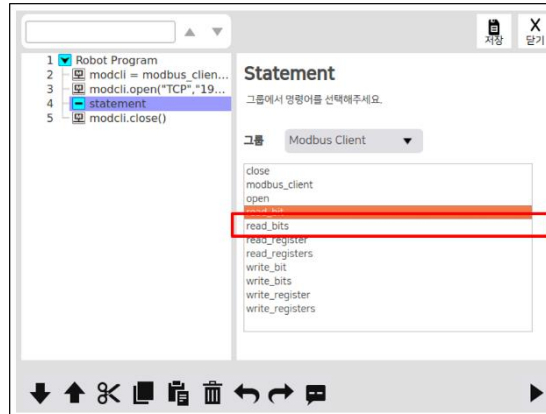
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 코일 값을 저장할 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 값을 읽어 올 코일의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.



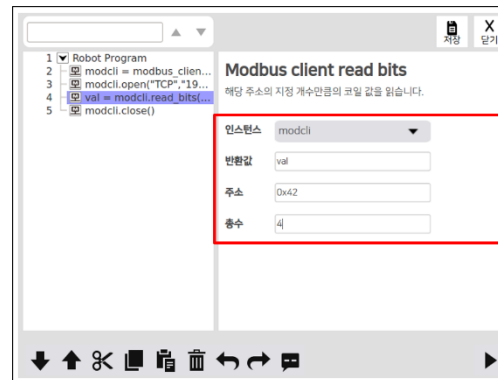
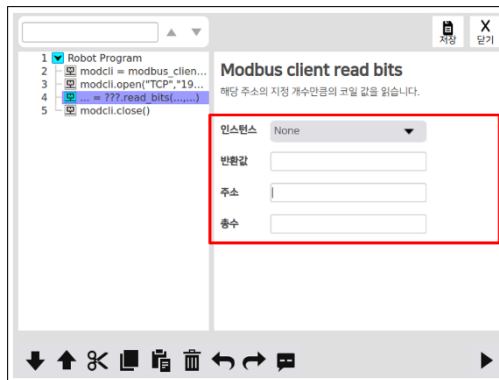
- read_bit 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

5. Read_bits

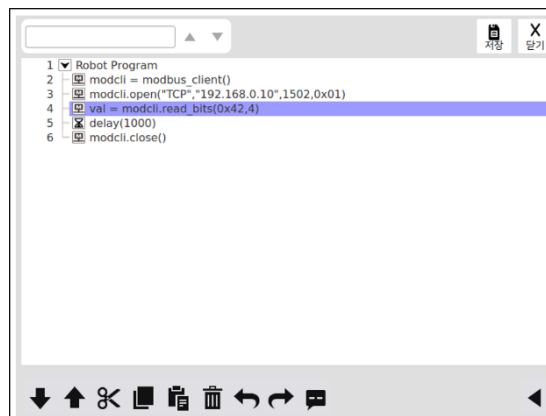
- read_bits 는 설정 주소의 지정 개수 만큼의 코일 값을 읽는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 read_bits 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



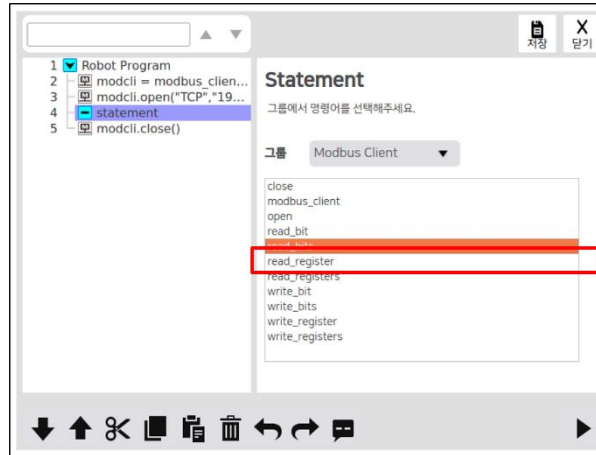
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 지정한 개수만큼의 코일 값을 저장할 테이블 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 값을 읽어 올 코일의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 총수는 읽어 올 코일 개수입니다. 입력 주소의 메모리에서 몇 개의 코일을 가져올 것인지 칸을 클릭하여 키보드를 통해 입력합니다.



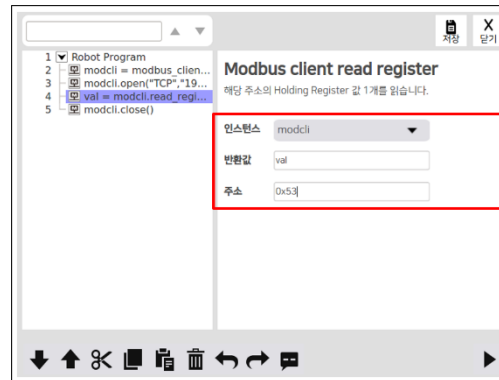
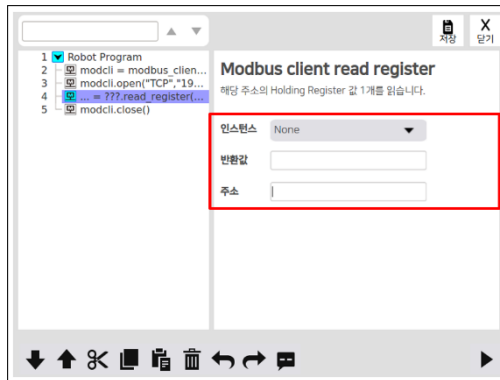
- read_bits 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

6. Read_register

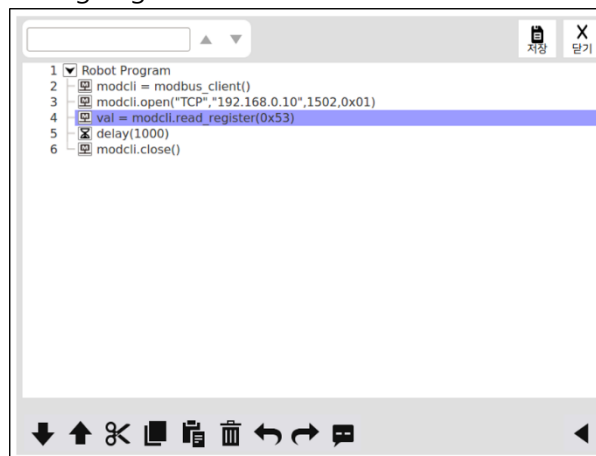
- read_register 는 설정 주소의 Holding Register 값 1 개를 읽는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 read_register 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



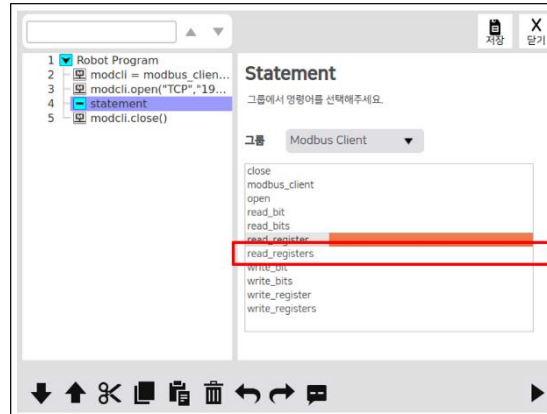
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 Holding Register 값을 저장할 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 값을 읽어 올 Holding Register 의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.



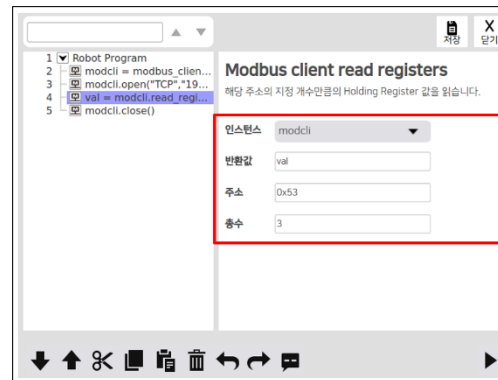
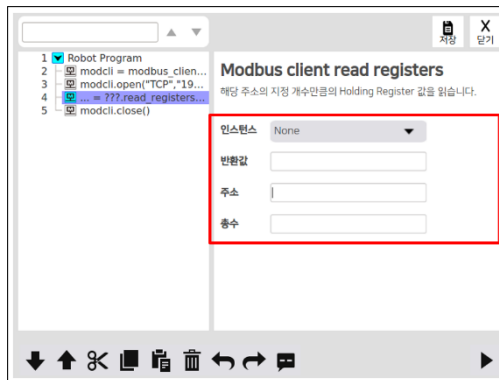
- read_register 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

7. Read_registers

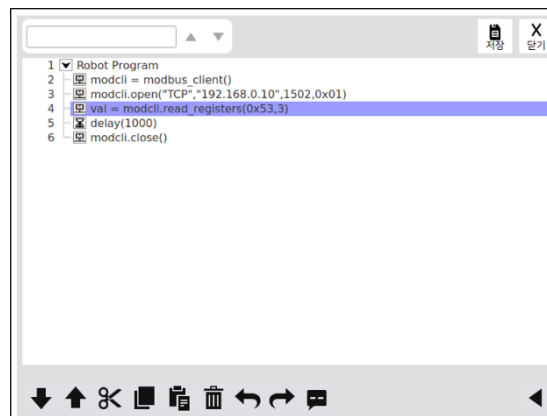
- read_registers 는 설정 주소의 지정 개수 만큼의 Holding Register 값을 읽는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 read_registers 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



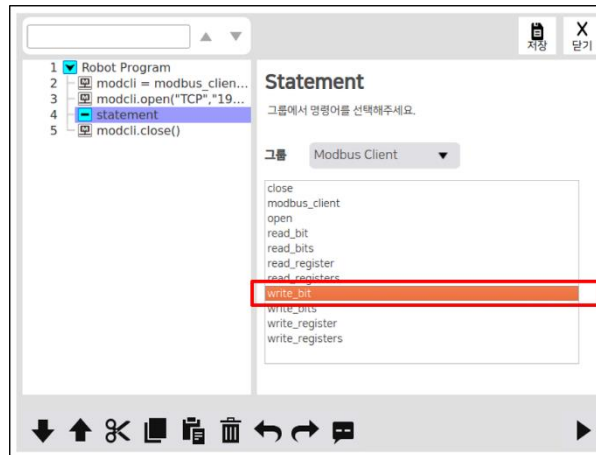
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 지정한 개수만큼의 Holding Register 값을 저장할 테이블 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 값을 읽어 올 Holding Register 의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 총수는 읽어 올 Holding Register 개수입니다. 입력 주소의 메모리에서 몇 개의 Holding Register 을 가져올 것인지 칸을 클릭하여 키보드를 통해 입력합니다.



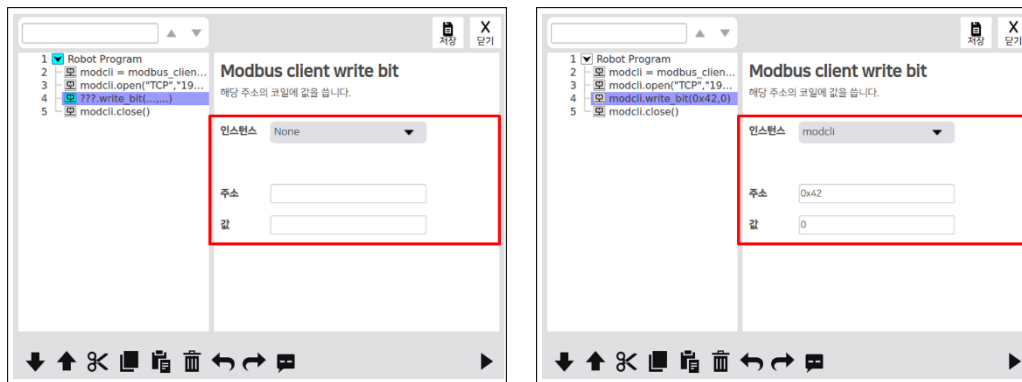
- read_registers 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

8. Write_bit

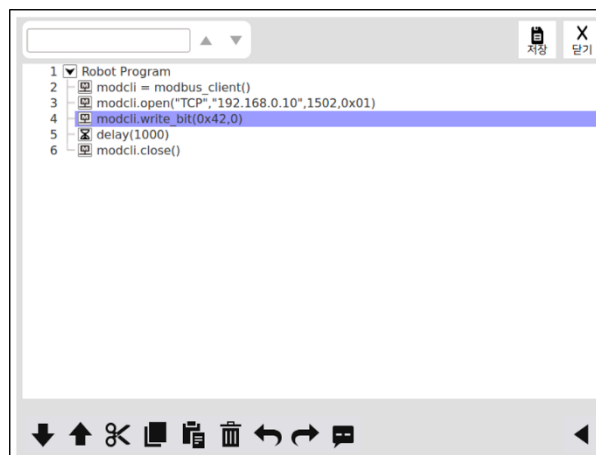
- write_bit 는 설정 주소의 코일 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 write_bit 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



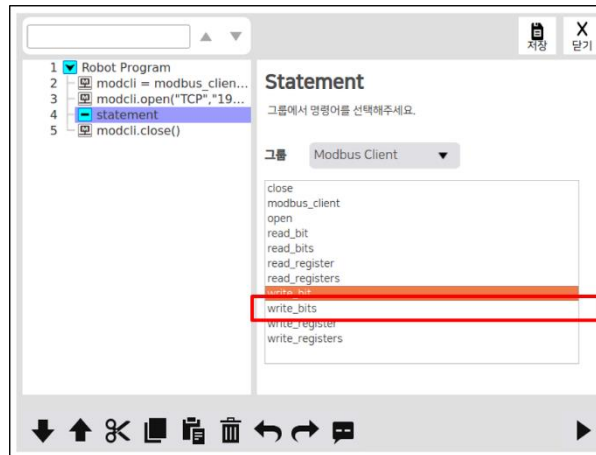
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 값을 쓸 코일의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 값은 설정 주소의 코일에 쓸 값 입니다. 칸을 클릭하여 키보드를 통해 입력합니다.



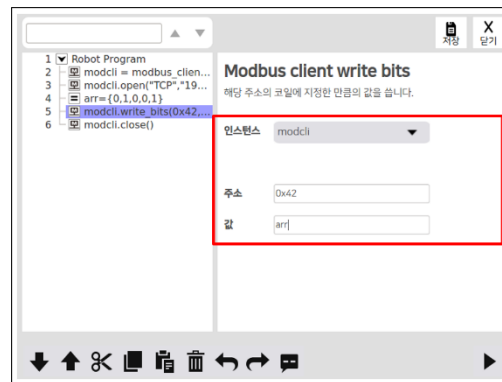
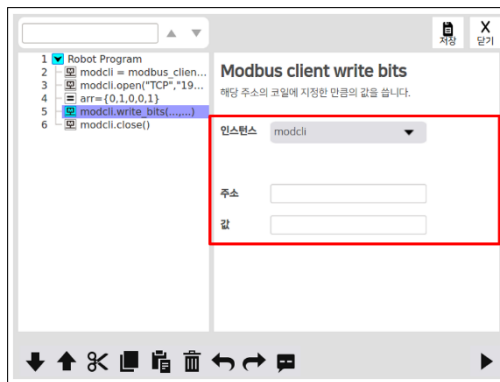
- write_bit 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

9. Write_bits

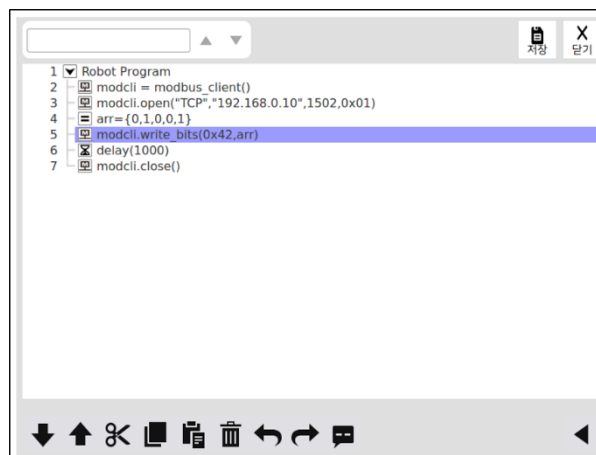
- write_bits 는 설정 주소의 코일에 지정한 만큼의 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 write_bits 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



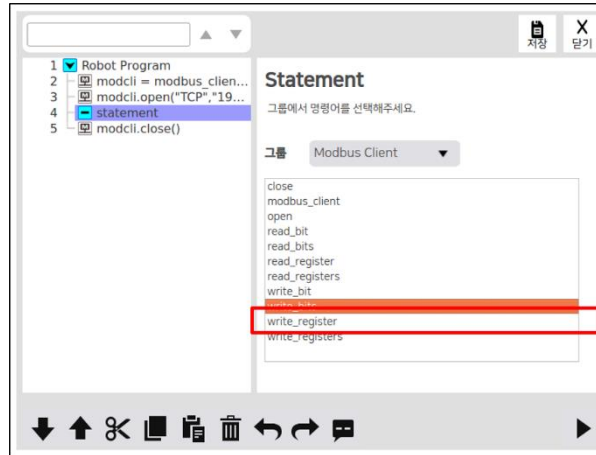
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 값을 쓸 코일의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 값은 설정 주소의 코일들에 쓸 값 입니다. 칸을 클릭하여 키보드를 통해 입력합니다.



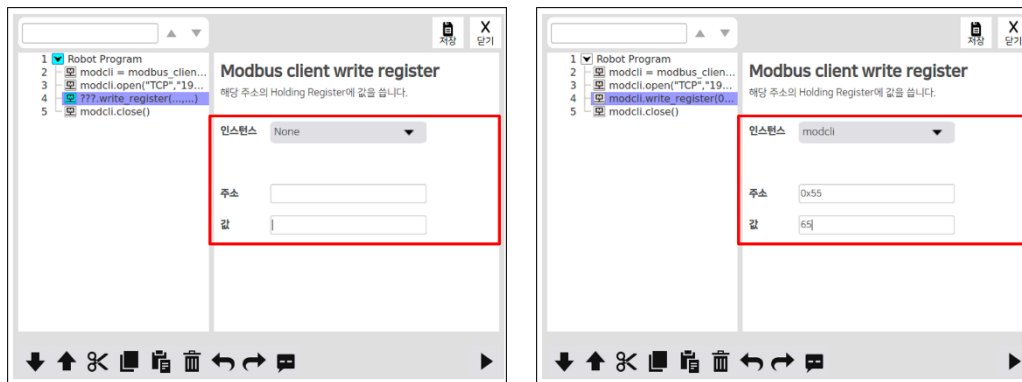
- write_bits 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

10. Write_register

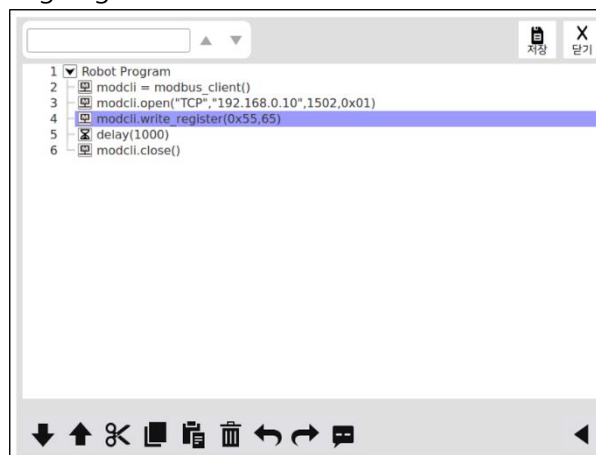
- write_register 는 설정 주소의 Holding Register 에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 write_register 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



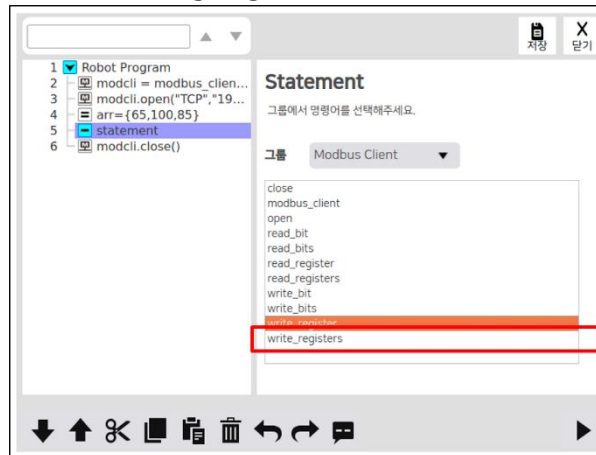
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 값을 쓸 Holding Register 의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 값은 설정 주소의 Holding Register 에 쓸 값 입니다. 칸을 클릭하여 키보드를 통해 입력합니다.



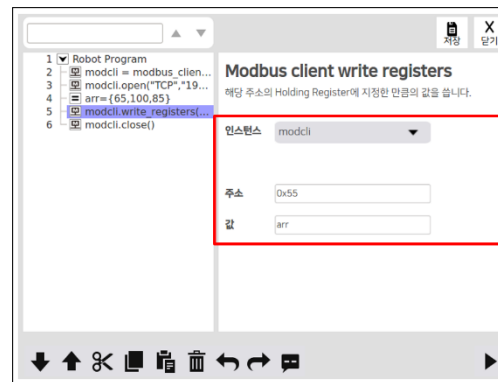
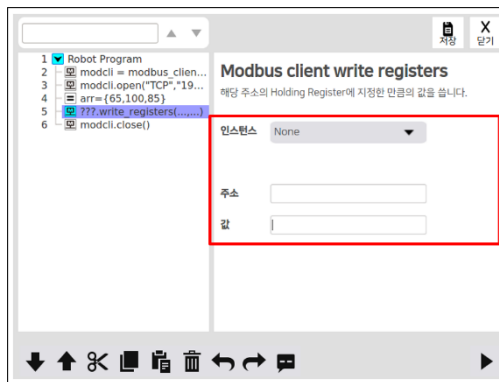
- write_register 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

11. Write_registers

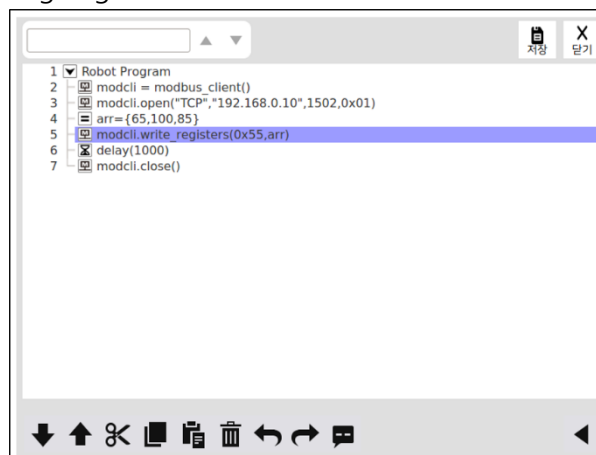
- write_registers 는 설정 주소의 Holding Register 에 지정한 만큼의 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Client 그룹에 있는 write_registers 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 값을 쓸 Holding Register 의 주소입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 값은 설정 주소의 Holding Register 들에 쓸 값 입니다. 칸을 클릭하여 키보드를 통해 입력합니다.

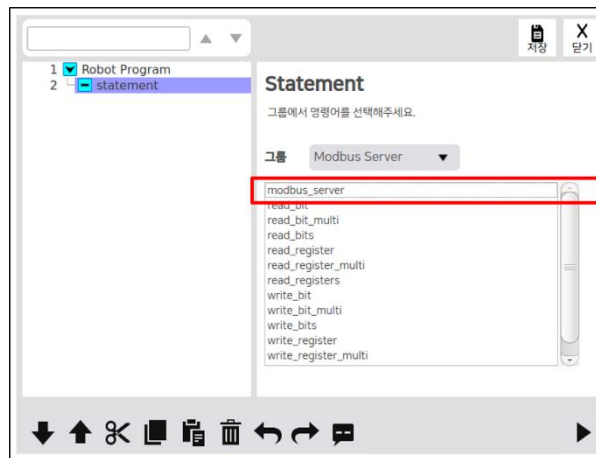


- write_registers 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

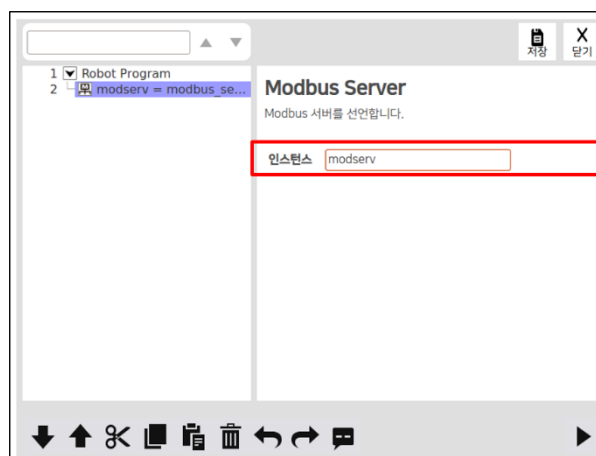
1.1.1.3.7 Modbus Server

1. Modbus_server()

- modbus_server 는 모드버스 서버를 선언하는 명령어 입니다.



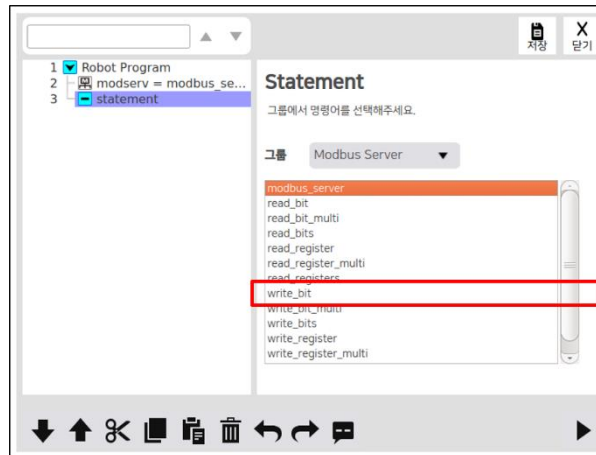
- Statement 의 Modbus Server 그룹에 있는 modbus_server 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



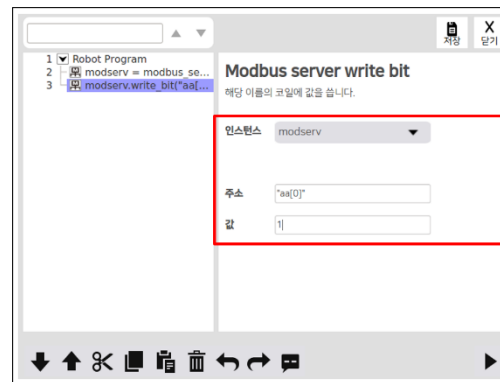
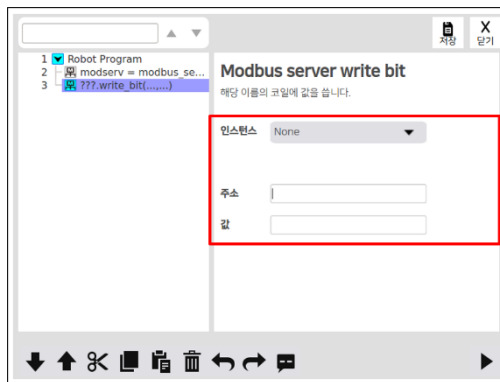
- 모드버스 서버를 선언할 인스턴스 명을 입력해야 합니다. 인스턴스 칸을 클릭하여 키보드를 통해 인스턴스 명을 입력합니다.

2. Write_bit

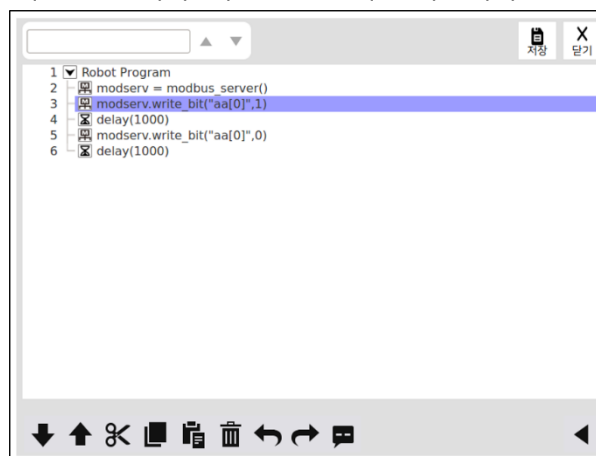
- write_bit 는 설정 이름의 코일에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 write_bit 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



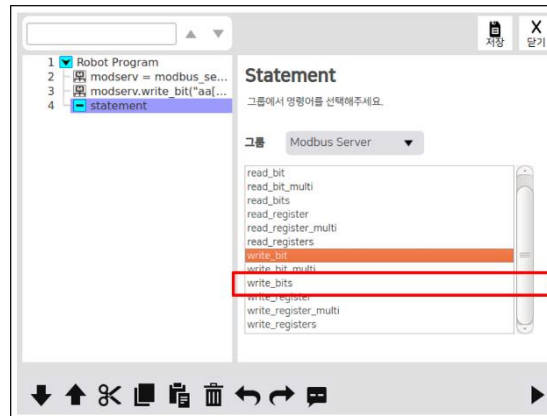
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 코일 이름으로 미리 선언된 코일의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다.
- 값은 코일 값으로 코일에 쓸 값입니다. 키보드를 통해 입력합니다.



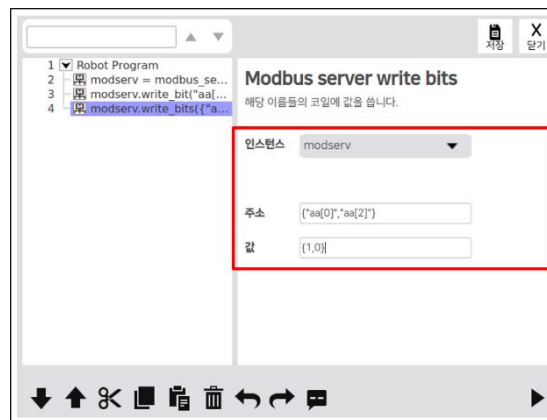
- write_bit 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

3. Write_bits

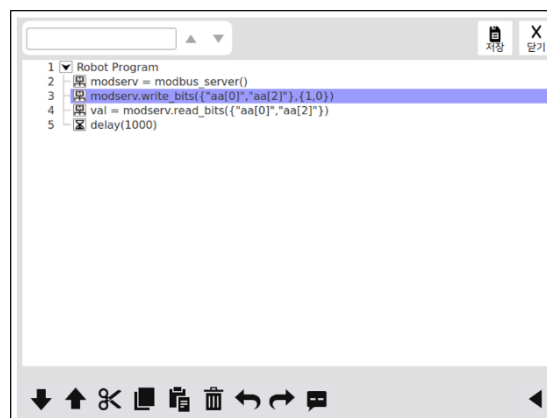
- write_bits 는 설정 이름들의 코일에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 write_bits 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



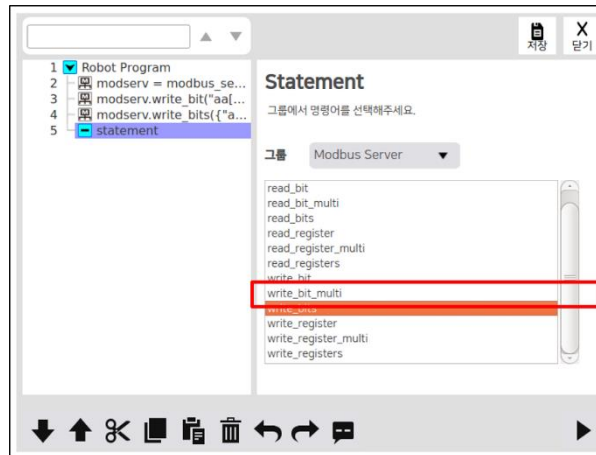
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 코일 이름으로 미리 선언된 코일의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다. 전체 코일 이름은 { } 괄호 사이에 입력합니다.
- 값은 코일 값으로 코일에 쓸 값입니다. 키보드를 통해 입력합니다. 전체 값은 { } 괄호 사이에 입력합니다.



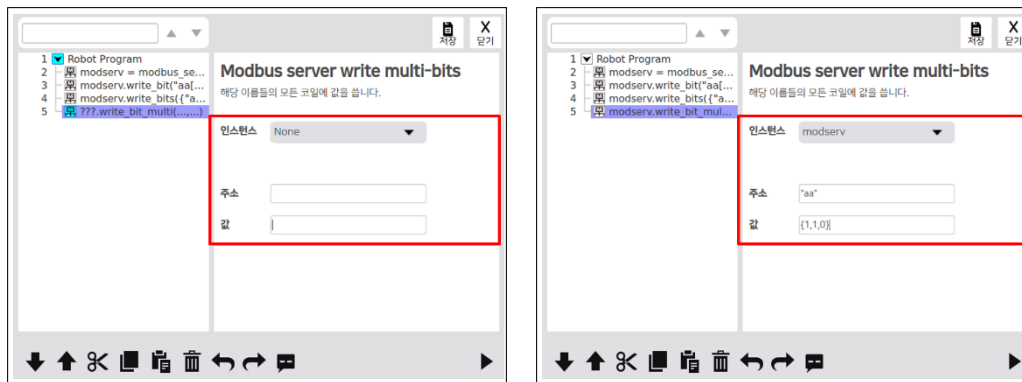
- write_bits 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

4. Write_bit_multi

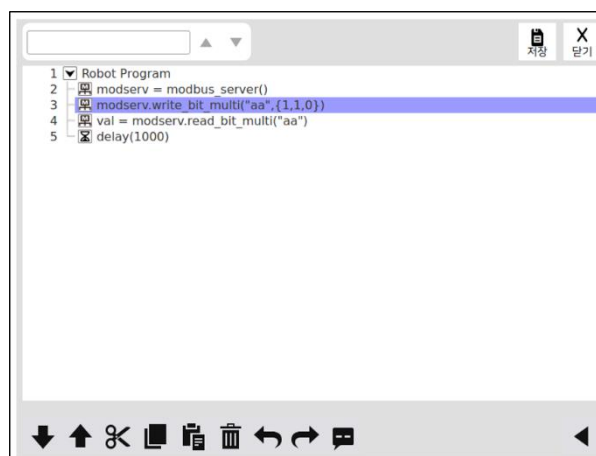
- write_bit_multi 는 설정 이름들의 모든 코일에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 write_bit_multi 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



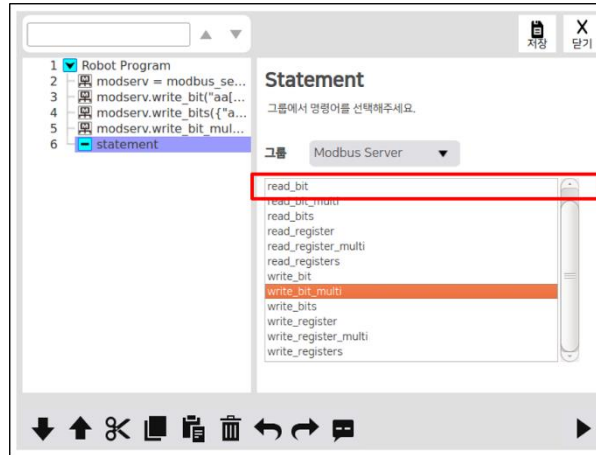
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 코일 이름으로 미리 선언된 코일의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다.
- 값은 코일 값으로 코일에 쓸 값입니다. 키보드를 통해 입력합니다. 전체 값은 { } 괄호 사이에 입력합니다.



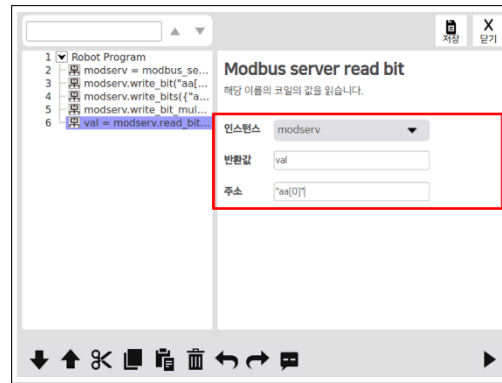
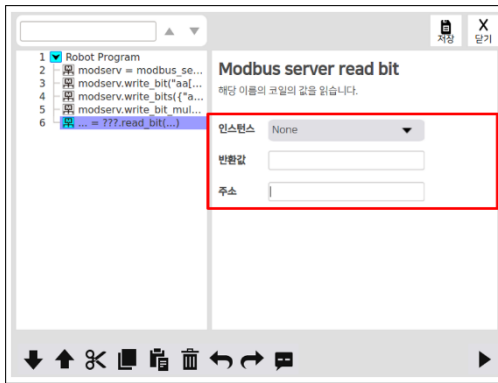
- write_bit_multi 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

5. Read_bit

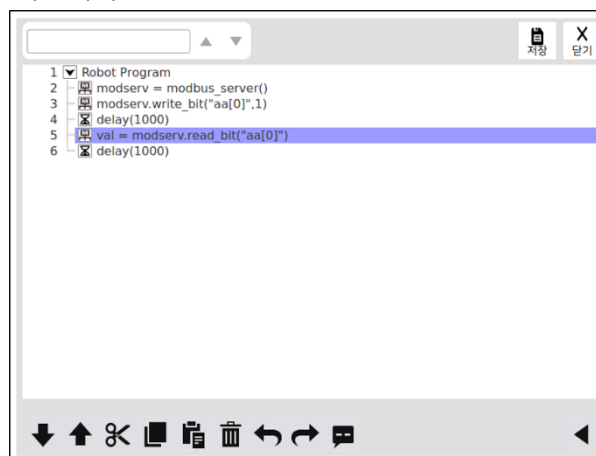
- read_bit 는 설정 이름의 코일 값을 읽는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 read_bit 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



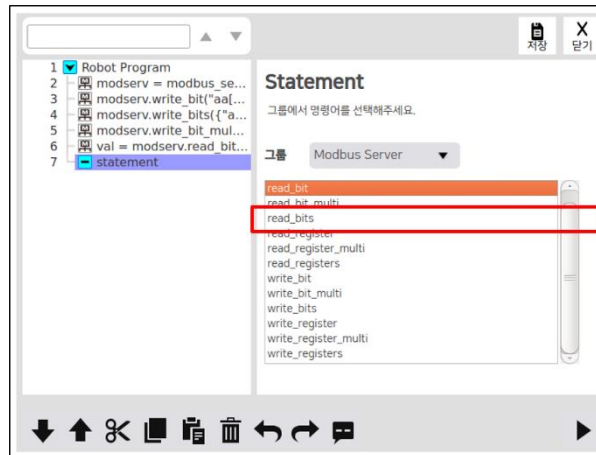
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 코일 값을 저장할 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 코일 이름으로 미리 선언된 코일의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다.



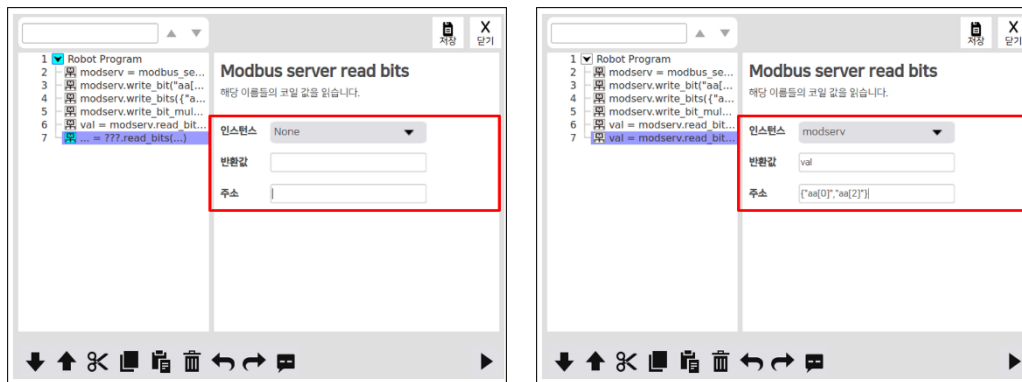
- read_bit 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

6. Read_bits

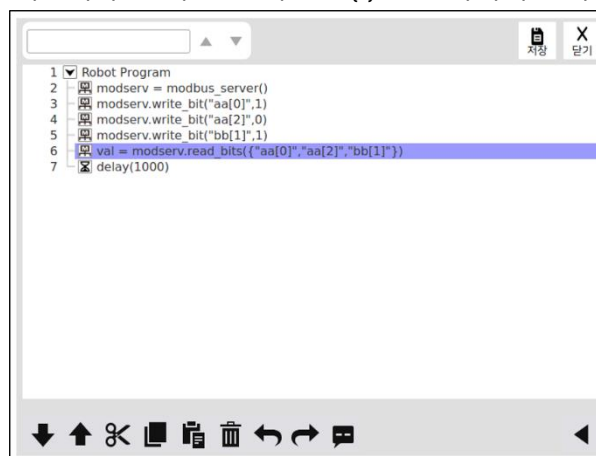
- read_bits 는 설정 이름들의 코일 값을 읽는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 read_bits 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



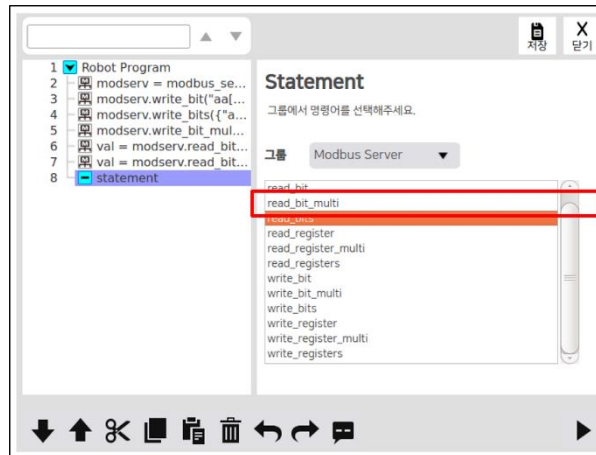
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 코일 값을 저장할 테이블 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 코일 이름으로 미리 선언된 코일의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다. 전체 코일 이름은 {} 괄호 사이에 입력합니다.



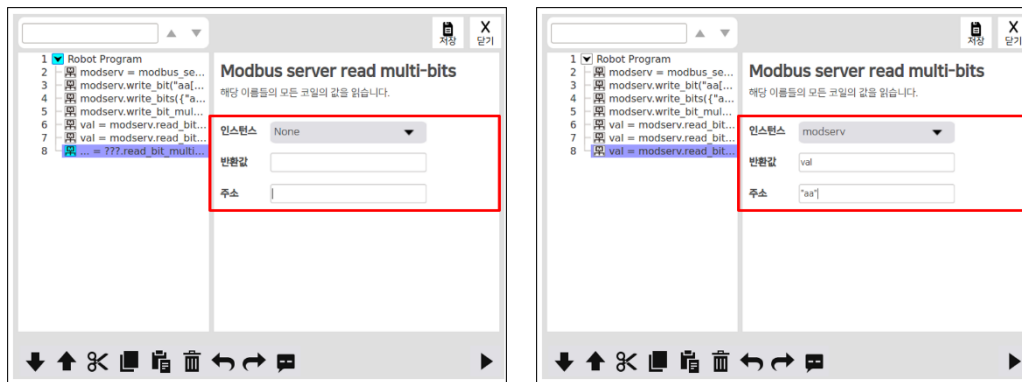
- read_bits 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

7. Read_bit_multi

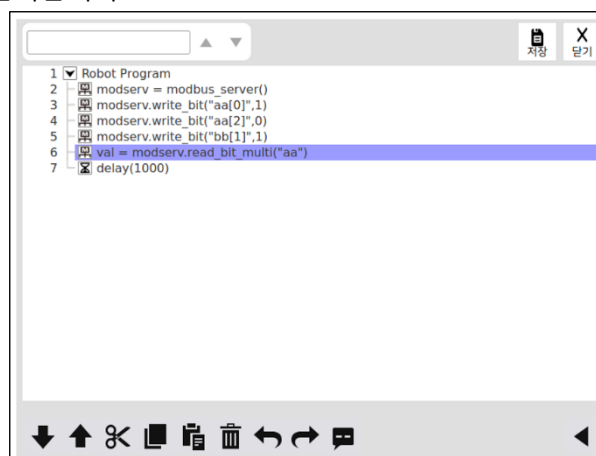
- read_bit_multi 는 설정 이름들의 모든 코일의 값을 읽는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 read_bit_multi 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



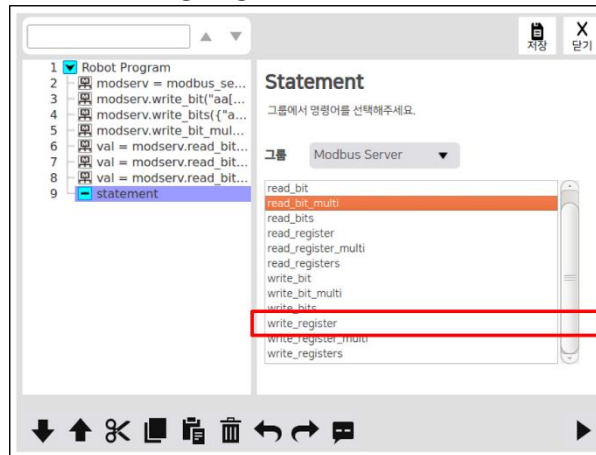
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 모든 코일 값을 저장할 테이블 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 코일 이름으로 미리 선언된 코일의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다.



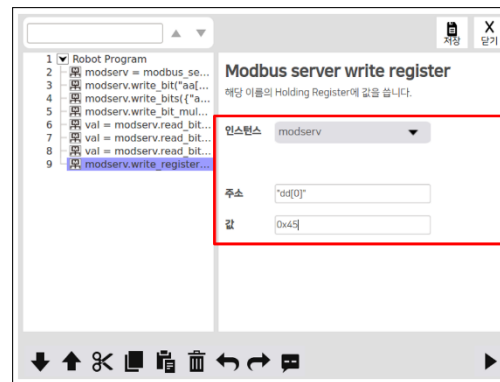
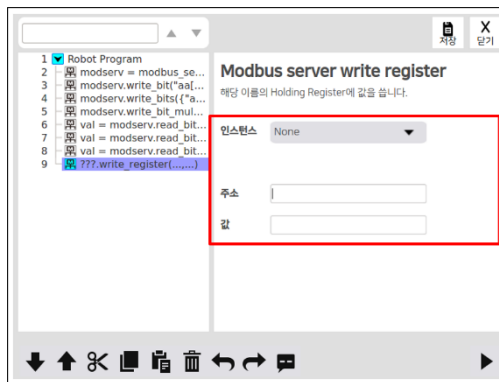
- read_bit_multi 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

8. Write_register

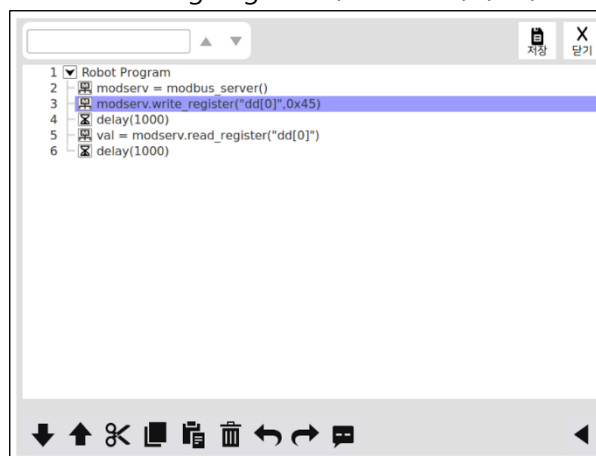
- write_register 은 설정 이름의 Holding Register 에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 write_register 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



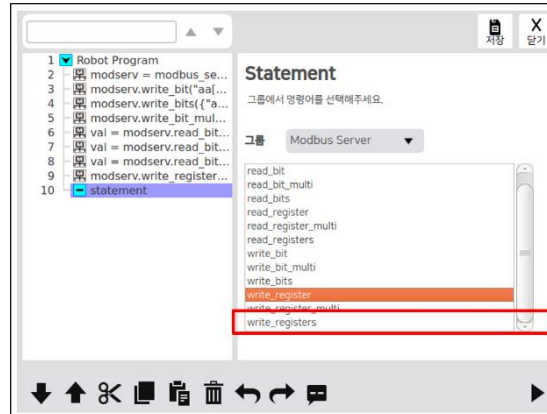
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 Holding Register 이름으로 미리 선언된 Holding Register 의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다.
- 값은 Holding Register 값으로 Holding Register 에 쓸 값입니다. 키보드를 통해 입력합니다.



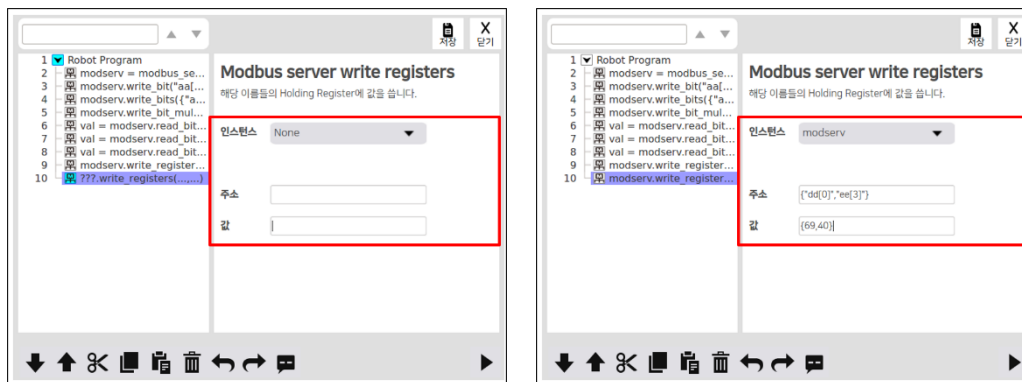
- write_register 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

9. Write_registers

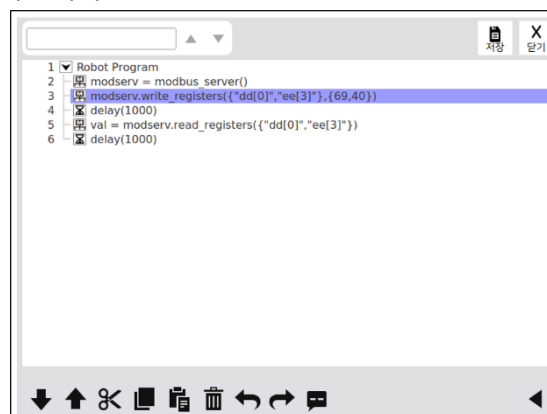
- write_registers 는 설정 이름들의 Holding Register 에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 write_registers 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



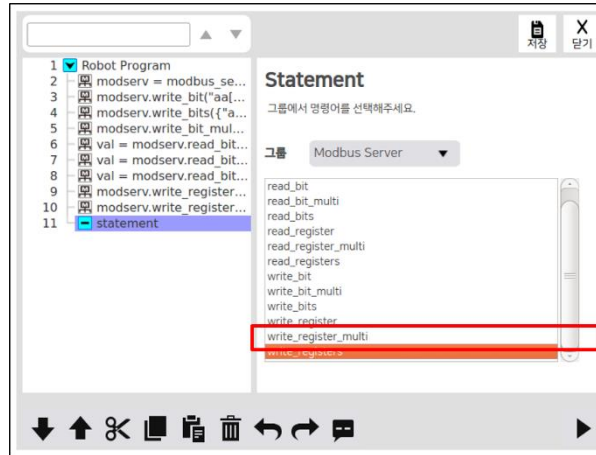
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 Holding Register 이름으로 미리 선언된 Holding Register 의 이름입니다. 키보드를 통해 입력합니다. Holding Register 의 이름은 ""(쌍따옴표) 사이에 입력합니다. 전체 Holding Register 이름은 { } 괄호 사이에 입력합니다.
- 값은 Holding Register 값으로 Holding Register 에 쓸 값입니다. 키보드를 통해 입력합니다. 전체 값은 { } 괄호 사이에 입력합니다.



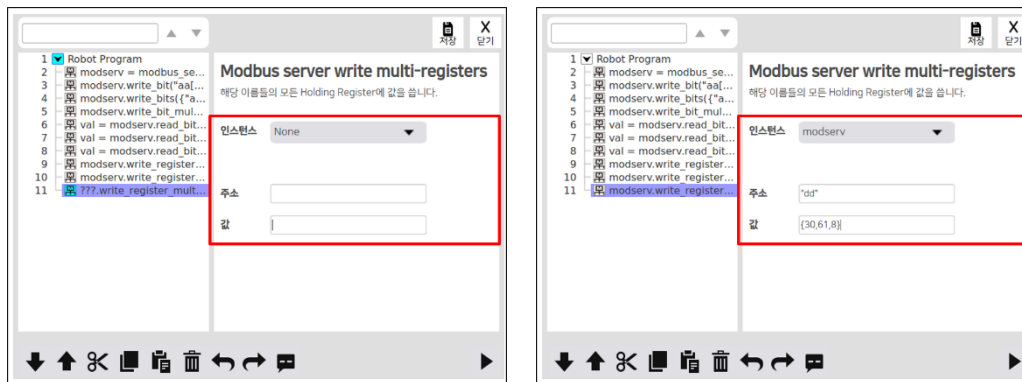
- write_registers 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

10. Write_register_multi

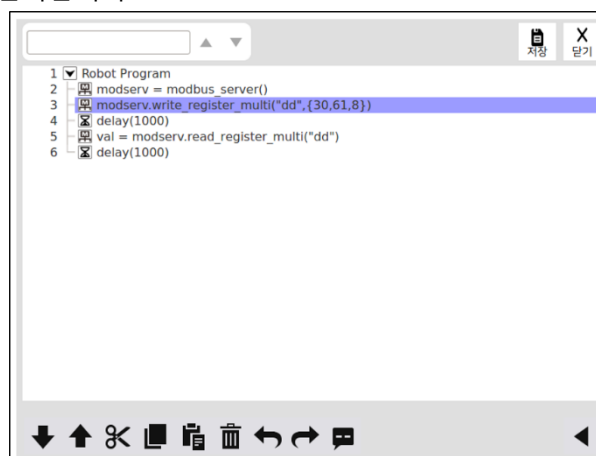
- write_register_multi 는 설정 이름들의 모든 Holding Register 에 값을 쓰는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 write_register_multi 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



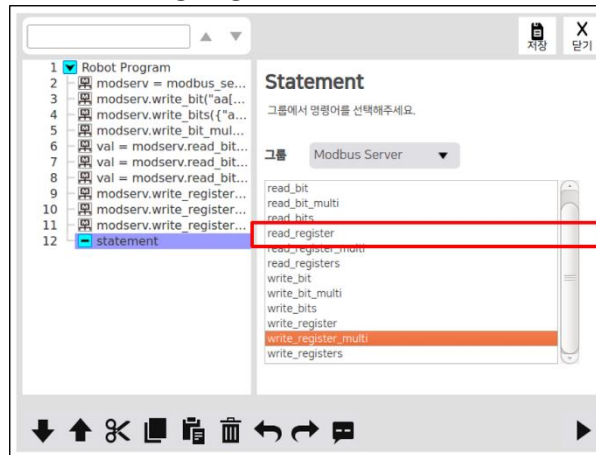
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 주소는 Holding Register 이름으로 미리 선언된 Holding Register 의 이름입니다. 키보드를 통해 입력합니다. 코일의 이름은 ""(쌍따옴표) 사이에 입력합니다.
- 값은 Holding Register 값으로 Holding Register 에 쓸 값입니다. 키보드를 통해 입력합니다. 전체 값은 {} 괄호 사이에 입력합니다.



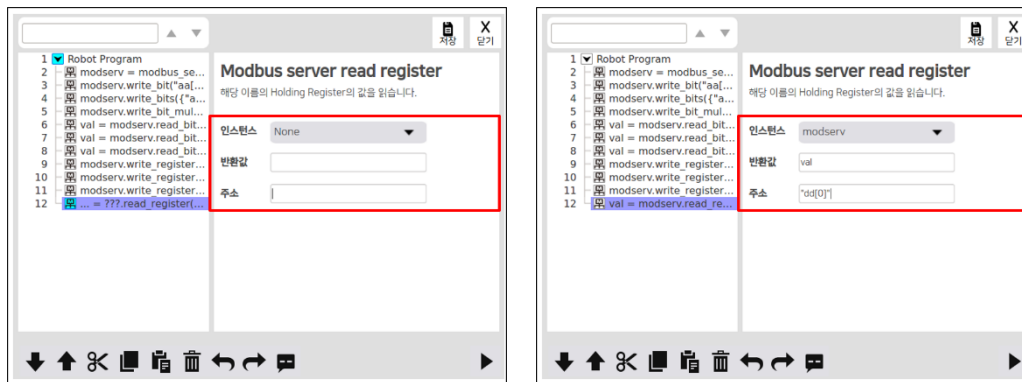
- write_register_multi 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

11. Read_register

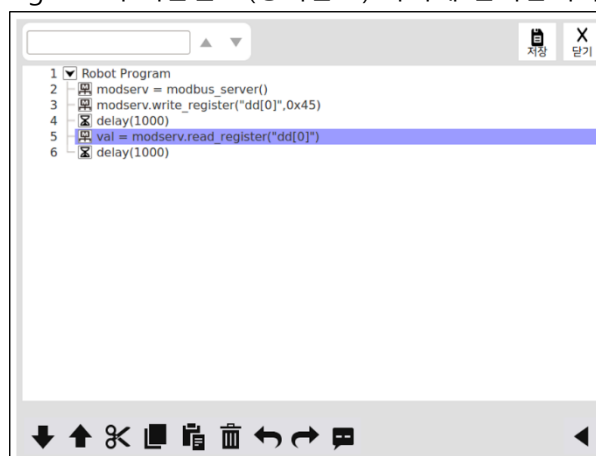
- read_register 는 설정 이름의 Holding Register 값을 읽는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 read_register 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



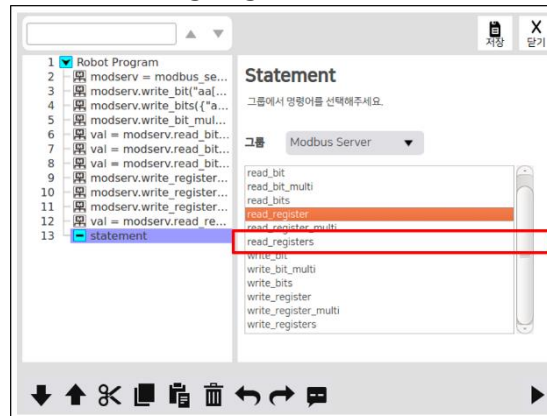
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 Holding Register 값을 저장할 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 Holding Register 이름으로 미리 선언된 Holding Register 의 이름입니다. 키보드를 통해 입력합니다. Holding Register 의 이름은 ""(쌍따옴표) 사이에 입력합니다.



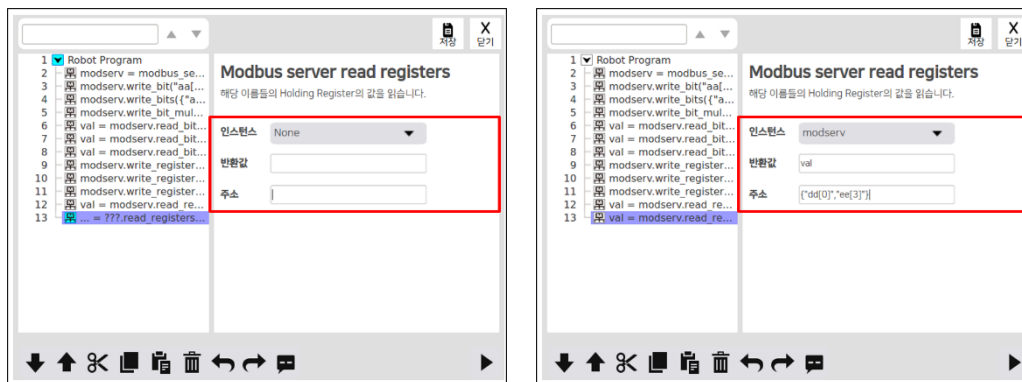
- read_register 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

12. Read_registers

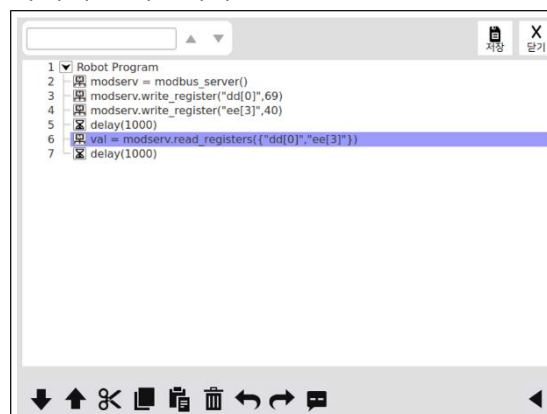
- read_registers 는 설정 이름들의 Holding Register 값을 읽는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 read_registers 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



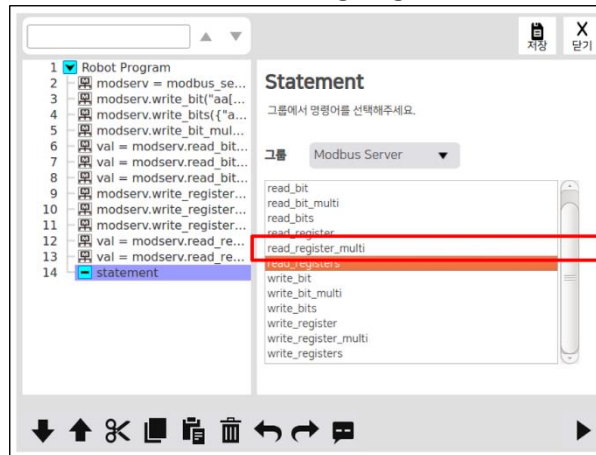
- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 Holding Register 값을 저장할 테이블 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 Holding Register 이름으로 미리 선언된 Holding Register 의 이름입니다. 키보드를 통해 입력합니다. Holding Register 의 이름은 ""(쌍따옴표) 사이에 입력합니다. 전체 Holding Register 이름은 { } 괄호 사이에 입력합니다.



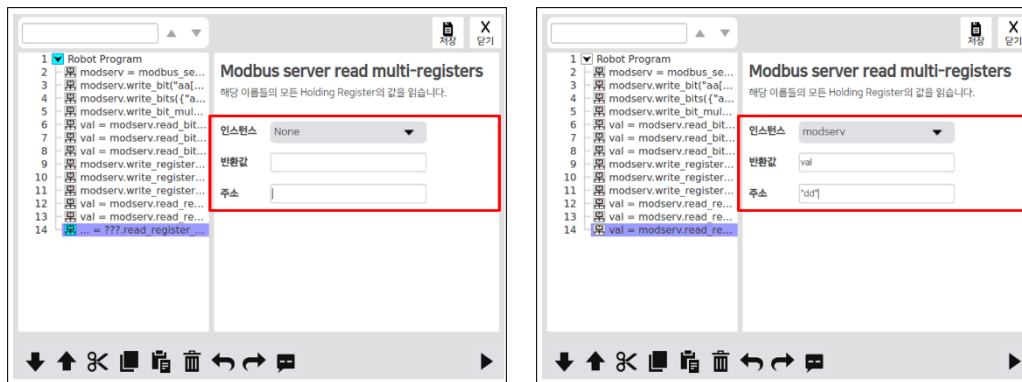
- read_registers 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

13. Read_register_multi

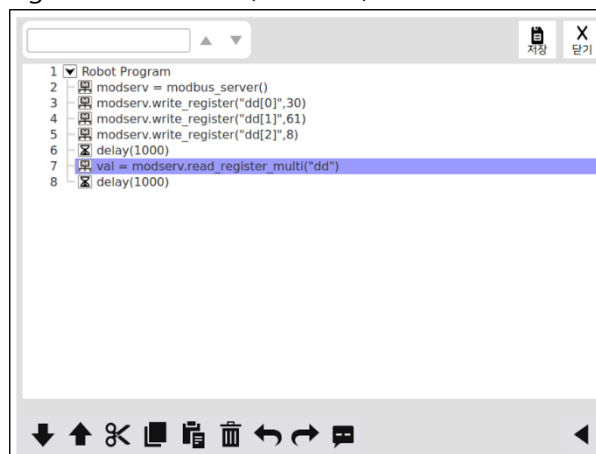
- read_register_multi 는 설정 이름들의 모든 Holding Register 의 값을 읽는 명령어 입니다.



- Statement 의 Modbus Server 그룹에 있는 read_register_multi 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- 인스턴스는 콤보 박스를 통해 설정합니다.
- 반환값은 읽어온 모든 Holding Register 값을 저장할 테이블 변수입니다. 칸을 클릭하여 키보드를 통해 입력합니다.
- 주소는 Holding Register 이름으로 미리 선언된 Holding Register 의 이름입니다. 키보드를 통해 입력합니다. Holding Register 의 이름은 ""(쌍따옴표) 사이에 입력합니다.

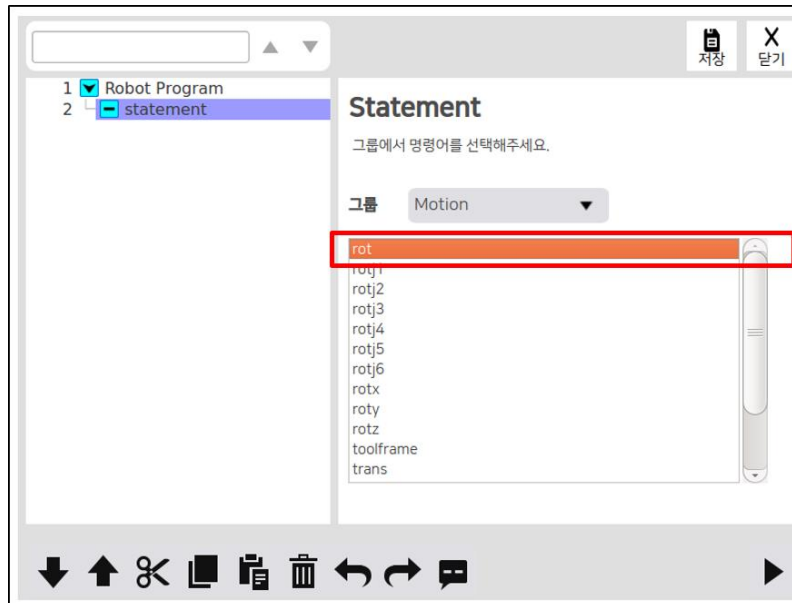


- read_register_multi 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

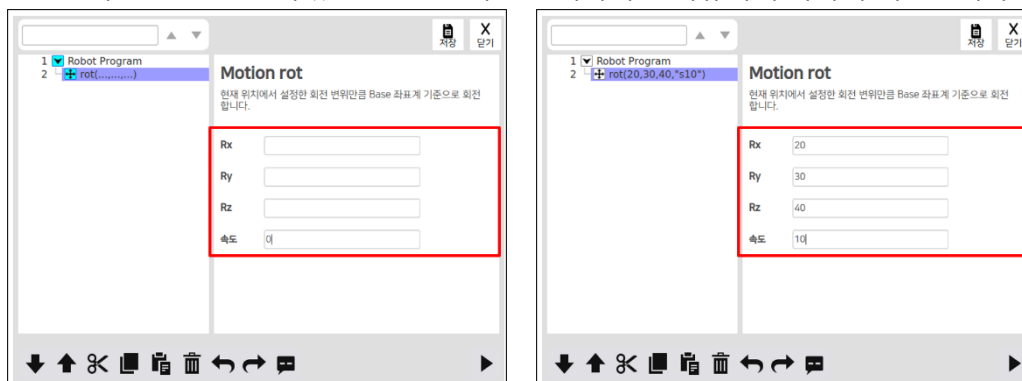
1.1.1.3.8 Motion

1. Rot

- rot 는 현재 위치에서 설정한 회전 변위 만큼 Base 좌표계 기준으로 회전하는 명령어 입니다.



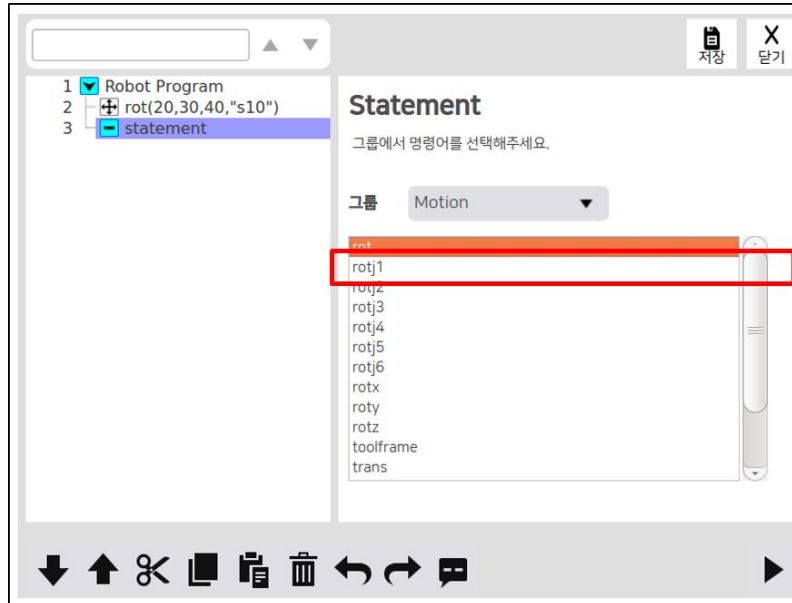
- Statement 의 Motion 그룹에 있는 rot 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



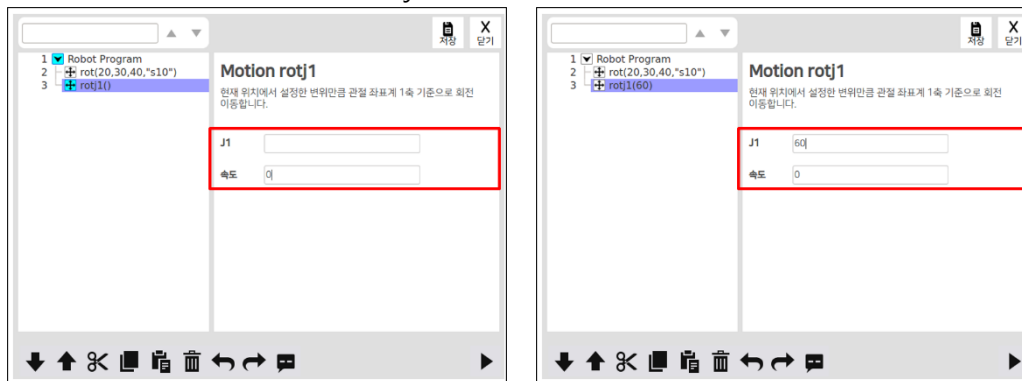
- 현재 위치에서 Base 좌표계의 축 중심으로 회전 이동할 변위의 칸을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.

2. Rotj1

- rotj1 는 현재 위치에서 설정한 변위만큼 관절 1 축 기준으로 회전 이동하는 명령어 입니다.



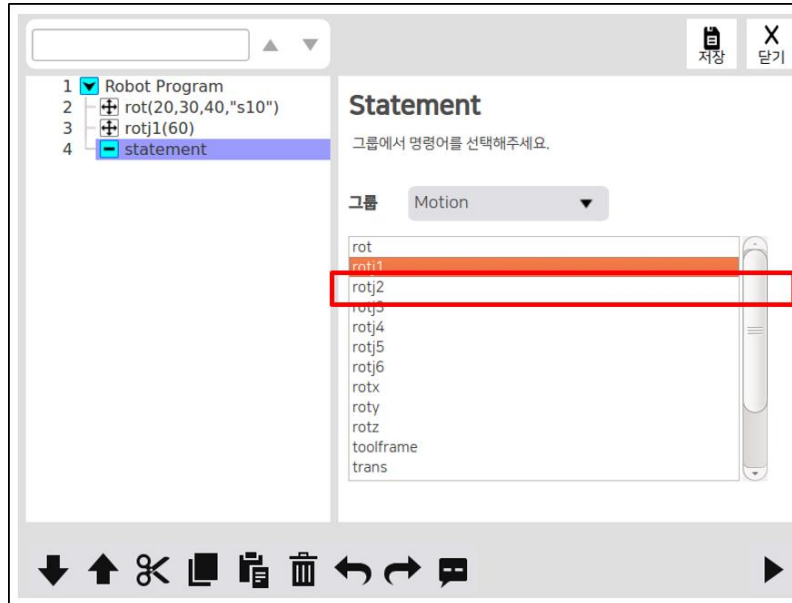
- Statement 의 Motion 그룹에 있는 rotj1 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



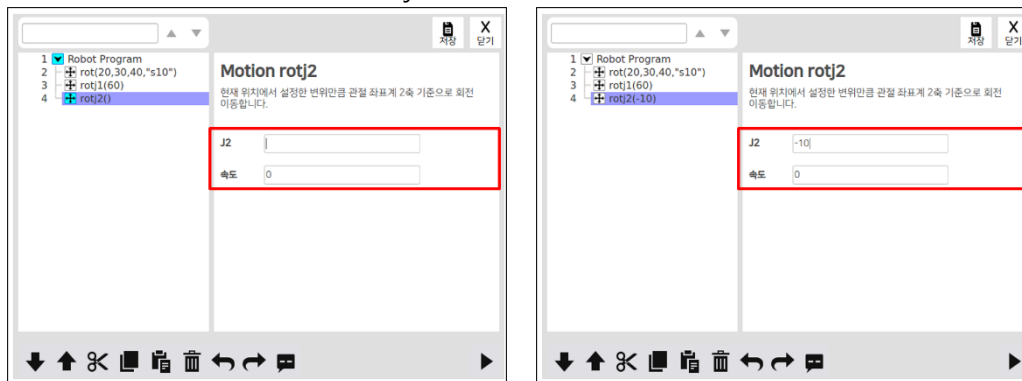
- J1 칸을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.

3. Rotj2

- rotj2 는 현재 위치에서 설정한 변위만큼 관절 2 축 기준으로 회전 이동하는 명령어 입니다.



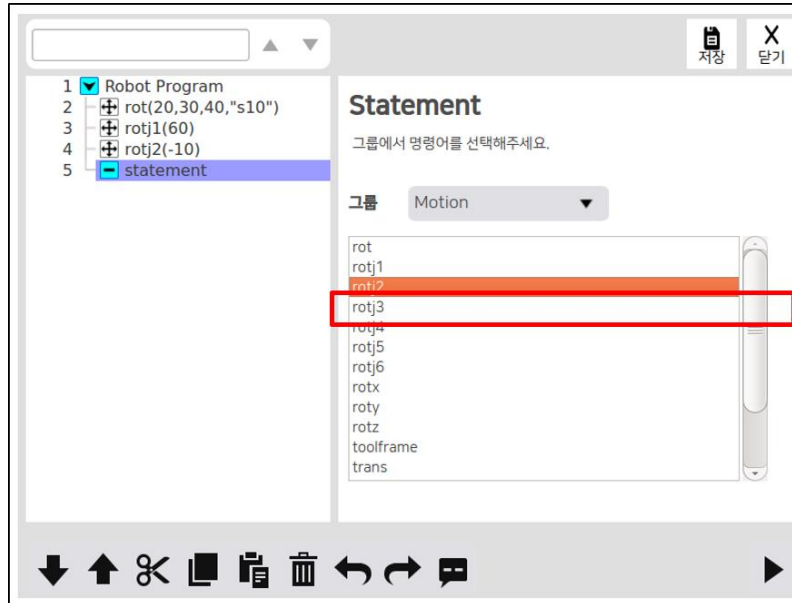
- Statement 의 Motion 그룹에 있는 rotj2 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



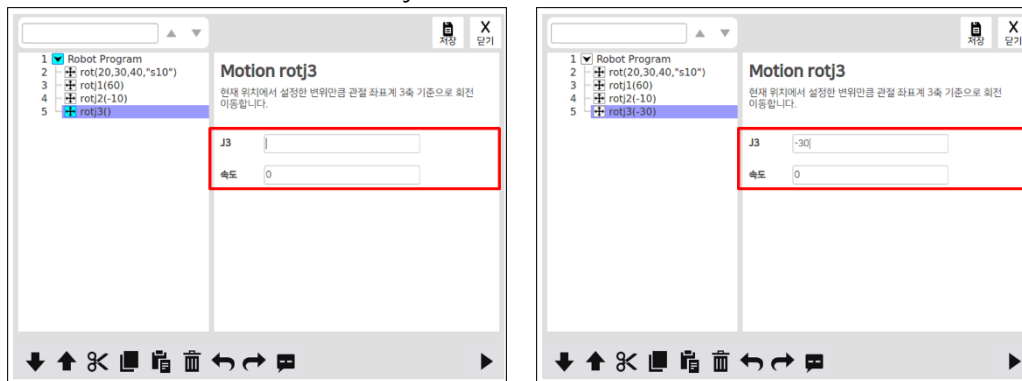
- J2 칸을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.

4. Rotj3

- rotj3 은 현재 위치에서 설정한 변위만큼 관절 3 축 기준으로 회전 이동하는 명령어 입니다.



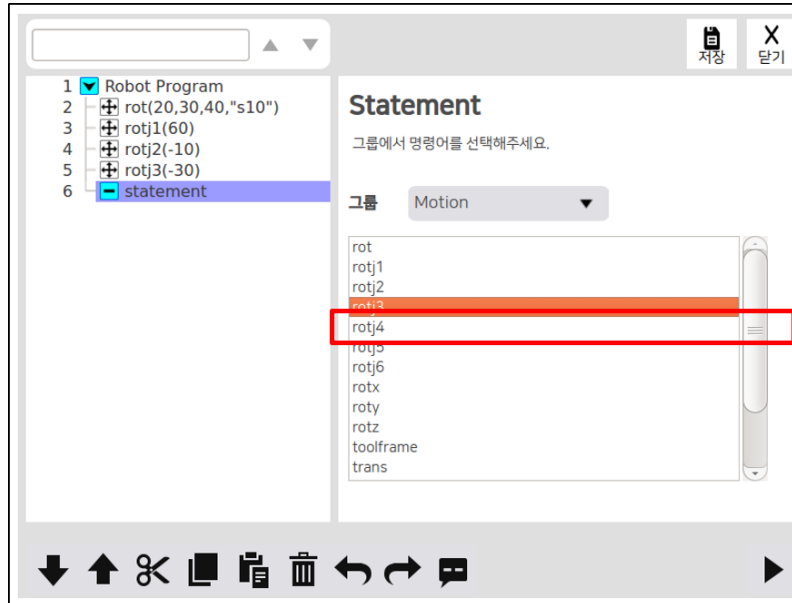
- Statement 의 Motion 그룹에 있는 rotj3 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



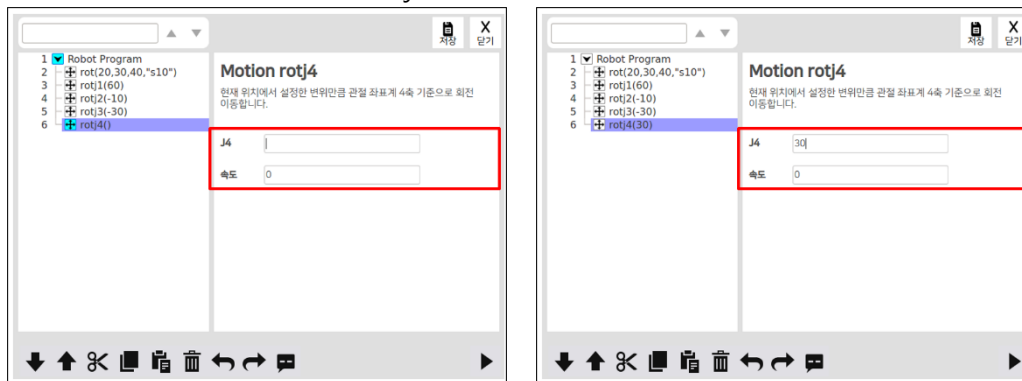
- J3 칸을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.

5. Rotj4

- rotj4 는 현재 위치에서 설정한 변위만큼 관절 4 축 기준으로 회전 이동하는 명령어 입니다.



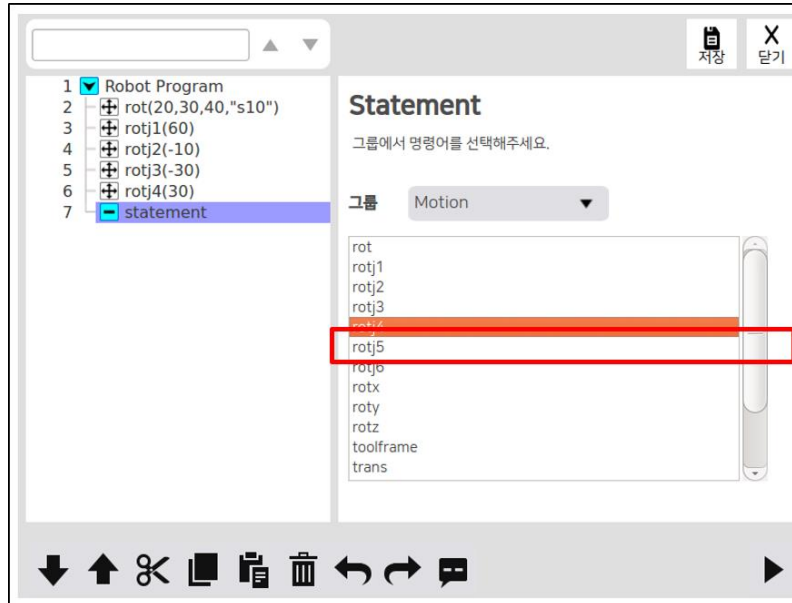
- Statement 의 Motion 그룹에 있는 rotj4 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



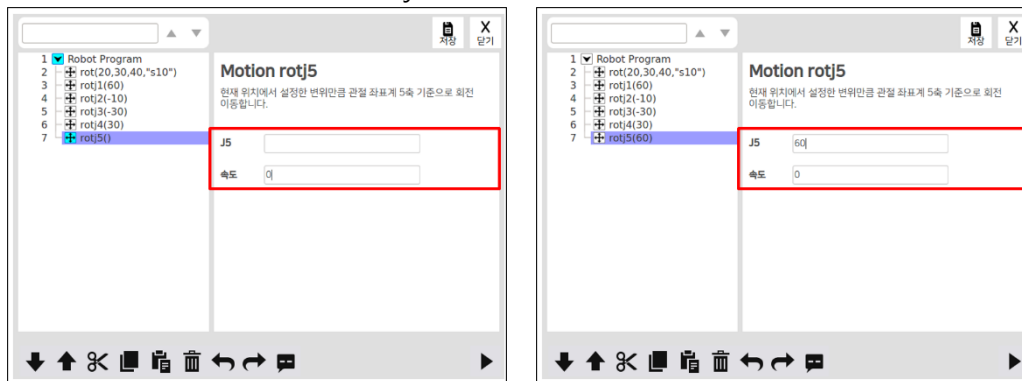
- J4 칸을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.

6. Rotj5

- rotj5 는 현재 위치에서 설정한 변위만큼 관절 5 축 기준으로 회전 이동하는 명령어 입니다.



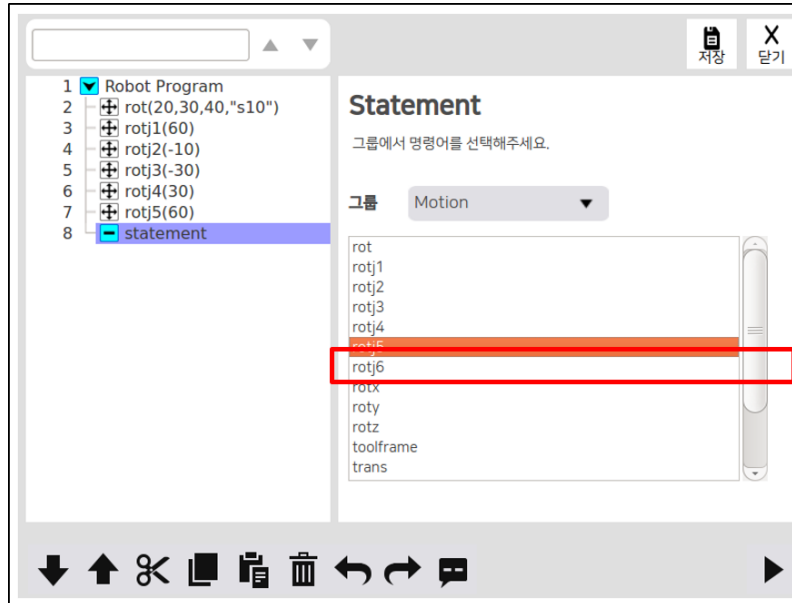
- Statement 의 Motion 그룹에 있는 rotj5 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



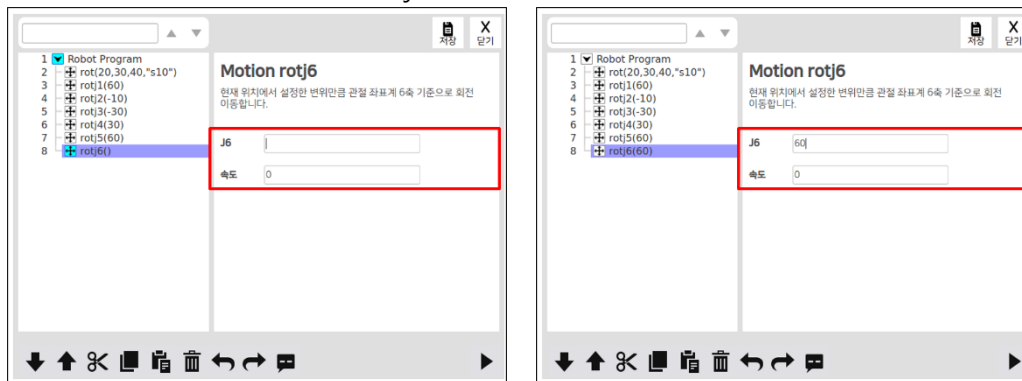
- J5 을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.

7. Rotj6

- rotj6 은 현재 위치에서 설정한 변위만큼 관절 6 축 기준으로 회전 이동하는 명령어 입니다.



- Statement 의 Motion 그룹에 있는 rotj6 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- J6 칸을 클릭하여 키보드 통해 값을 입력합니다.
- 속도는 상대 속도 설정으로 기본적으로 0 이 입력되어 있습니다. 0 은 상대 속도 설정이 아닌 시스템 속도로 설정됩니다. 구간마다 로봇의 속도를 조절하는 경우 사용합니다. 사용 시, 속도 칸을 클릭하여 키보드를 통해 값을 입력합니다.